Journal of Computational Physics 559 (2026) 114894

Contents lists available at ScienceDirect

Journal of Computational Physics

journal homepage: www.elsevier.com/locate/jcp

Learning singularity-encoded Green's functions with application to iterative methods ⋆

Qi Sun [a,b,*], Shengyan Li [a], Bowen Zheng [c,d], Lili Ju [e], Xuejun Xu [a,b]

[a] School of Mathematical Sciences, Tongji University, Shanghai, 200092, China
[b] Key Laboratory of Intelligent Computing and Applications (Ministry of Education), Tongji University, Shanghai, 200092, China
[c] LSEC, ICMSEC, Academy of Mathematics and Systems Science, Chinese Academy of Sciences, Beijing, 100190, China
[d] School of Mathematical Sciences, University of Chinese Academy of Sciences, Beijing, 100190, China
[e] Department of Mathematics, University of South Carolina, Columbia, SC, 29208, USA

ARTICLE INFO

Keywords:
Elliptic problem
Singularity-encoded Green's function
Neural network
Preconditioned iterative solver
Spectral bias

ABSTRACT

Green's function provides an inherent connection between theoretical analysis and numerical methods for elliptic partial differential equations, and general absence of its closed-form expression necessitates surrogate modeling to guide the design of effective solvers. Unfortunately, numerical computation of Green's function remains challenging due to its doubled dimensionality and intrinsic singularity. In this paper, we present a novel singularity-encoded learning approach to resolve these problems in an unsupervised fashion. Our method embeds the Green's function within a one-order higher-dimensional space by encoding its prior estimate as an augmented variable, followed by a neural network parametrization to manage the increased dimensionality. By projecting the trained neural network solution back onto the original domain, our deep surrogate model exploits its spectral bias to accelerate conventional iterative schemes, serving either as a preconditioner or as part of a hybrid solver. The effectiveness of our proposed method is empirically verified through numerical experiments with two and four dimensional Green's functions, achieving satisfactory resolution of singularities and acceleration of iterative solvers.

1. Introduction

Machine learning has achieved remarkable success across a wide range of engineering applications and scientific disciplines, emerging as an innovative methodology toward modern scientific computing [1–4]. Owing to its universal approximation capability and mesh-free discretization [5,6], the neural network-based method has become increasingly popular for the numerical solution of partial differential equations [7,8], especially in high dimensions [9] where classical mesh-based methods exhibit limitations. Scientific machine learning approaches within this field can be broadly categorized into two paradigms: solution learning and operator learning.

To be specific, we consider the benchmark elliptic boundary value problem of the form [7,10]

$$\begin{aligned} -\nabla\cdot(c(\boldsymbol{x})\nabla u(\boldsymbol{x})) &= f(\boldsymbol{x}), && \text{for } \boldsymbol{x}\in\Omega,\\ u(\boldsymbol{x}) &= 0, && \text{for } \boldsymbol{x}\in\partial\Omega, \end{aligned} \tag{1}$$

⋆ Q. Sun's work is partially supported by National Natural Science Foundation of China under grant number 12201465 and Science and Technology Commission of Shanghai Municipality under grant number 23JC1400502. X. Xu's work is partially supported by National Natural Science Foundation of China under grant numbers 12071350 and 12331015.

* Corresponding author.
E-mail addresses: qsun_irl@tongji.edu.cn (Q. Sun), 2410285@tongji.edu.cn (S. Li), zhengbowen@lsec.cc.ac.cn (B. Zheng), ju@math.sc.edu (L. Ju), xuxj@tongji.edu.cn (X. Xu).

https://doi.org/10.1016/j.jcp.2026.114894
Received 30 August 2025; Received in revised form 15 February 2026; Accepted 29 March 2026
Available online 1 April 2026
0021-9991/© 2026 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

where $\Omega \subset \mathbb{R}^d$ ($d \in \mathbb{N}_+$) is a bounded domain with a $C^{2,\alpha}$ boundary $\partial\Omega$ for some $\alpha \in (0,1)$, $c \in C^{1,\alpha}(\bar{\Omega})$ its coefficient function that is strictly positive and uniformly elliptic, and $f \in L^2(\Omega)$ the source term. Throughout this paper, the vector variable $\boldsymbol{x} = (x_1, \dots, x_d)$ is notationally simplified to a scalar variable $x \in \mathbb{R}$ when $d = 1$. In solution learning methods such as the physics-informed neural networks [6], the deep Ritz method [11], and the deep Galerkin method [12], the solution of problem (1) is parametrized using a neural network and optimized via minimizing either the residual of governing equations or the energy functional. These approaches are designed in an instance-specific manner, requiring retraining for any changes in the forcing term or boundary conditions.

On the other hand, operator learning aims to approximate mappings between function spaces [4,13,14], such as the solution operator of problem (1) that maps a forcing term to its corresponding solution, namely,

$$u(\boldsymbol{x}) = \int_\Omega G(\boldsymbol{x}, \boldsymbol{y}) f(\boldsymbol{y})\, d\boldsymbol{y}, \tag{2}$$

in terms of the $2d$-dimensional Green's function $G(\boldsymbol{x}, \boldsymbol{y}) : \Omega \times \Omega \to \mathbb{R} \cup \{\infty\}$ [7,10]. Such an analytic expression has motivated the construction of neural network models [14–18] that directly learn the integral operator (2) in a data-driven manner, while alternative approaches [13,19] build on the universal approximation theorems. In addition to serving as fast solvers, neural operators are being seamlessly integrated into conventional scientific computing workflows to enhance computational efficiency [4]. The mesh-free nature enables their deployment as effective preconditioning strategies for the discrete system of Eq. (1) across different mesh sizes [20], while their inherent spectral bias facilitates hybridization with classical iterative methods [21–23].

Although end-to-end neural operators have recently attracted considerable scientific interest, their reliance on extensive high-quality training datasets and limited physical interpretability remain challenging [4]. An emerging alternative is to learn the Green's function directly, either from discrete source-to-solution data pairs [15,16,24] or by exploiting its governing equations through physics-informed machine learning [24–28]. Data-driven methods, as previously noted, often require extensive labelled datasets to attain robust generalization, making the unsupervised learning approach preferable in practice [29]. To avoid the computation of the Dirac delta function, methods such as the Gaussian approximation [26,27,30], the fundamental solution-based subtraction [25,28] and the variational reformulation [31] are developed. On the other hand, the low regularity of Green's functions makes it necessary to establish specialized neural network architectures [24,30,32,33] to ensure approximation accuracy. Specifically, the Green's function of our elliptic problem (1) exhibits a piecewise smooth singularity when $d = 1$, a logarithmic singularity when $d = 2$, and an algebraic singularity when $d \geq 3$ [34–37], which is inherent to the kernel function that persists regardless of data-driven or physics-informed learning approaches. Notably, in scenarios where governing equations are unavailable, data-driven rational approximations can be applied to construct effective surrogates for the Green's function and to uncover its singularity structure [15,32,38].

In this paper, we present a novel singularity-encoded learning framework to reconstruct the Green's function without supervised training data. Building upon the analytic estimates of its singular behavior [34,35,37,39], the Green's function is embedded into a higher-dimensional space by incorporating an augmented variable, followed by the parametrization via neural networks to handle the increased dimensionality. It is noteworthy that the Dirac delta function is directly computed through integration along the singular region, rather than relying on Gaussian approximations or fundamental solutions. Besides, our explicit encoding of singularities enables a smooth neural network to reconstruct singular functions. The surrogate model for our Green's functions can then be obtained by projecting the trained network solution back onto the physical domain. By exploiting the spectral bias inherent in the training process, our singularity-encoded Green's function is utilized to accelerate classical iterative methods, either as a preconditioning matrix or a component in our hybrid solvers.

The remainder of this paper is organized as follows. Section 2 presents a comprehensive overview of existing methods for learning the Green's function via neural networks, together with the employment of neural operators in accelerating classical iterative methods. Our singularity-encoded learning method is presented in Section 3 and is illustrated through a one-dimensional benchmark problem for visualization. In Section 4, the pre-trained model is adopted to accelerate classical iterative schemes, serving either as a preconditioner or part of our hybrid solver. Numerical validation through extra experiments are provided in Section 5, followed by conclusions in Section 6.

2. Related work

2.1. Learning Green's functions with neural networks

For any impulse source located at $\boldsymbol{y} \in \Omega$, the Green's function associated with problem (1) satisfies

$$\begin{aligned} -\nabla \cdot (c(\boldsymbol{x}) \nabla G(\boldsymbol{x}, \boldsymbol{y})) &= \delta(\boldsymbol{x} - \boldsymbol{y}), && \text{for } \boldsymbol{x} \in \Omega, \\ G(\boldsymbol{x}, \boldsymbol{y}) &= 0, && \text{for } \boldsymbol{x} \in \partial\Omega, \end{aligned} \tag{3}$$

where $\delta(\boldsymbol{x})$ denotes the Dirac delta function. Unfortunately, the analytic expression of Green's function is often difficult to obtain, especially for problems with complex geometries or variable coefficients [7,39]. Alternatively, numerical approximation of Green's function becomes requisite, but conventional mesh-based methods, such as the finite difference or finite element methods [8,40], are confronted with some long-standing challenges [25] as follows:

- the doubled dimensionality of Green's function incurs the curse of dimensionality;
- the complex domain makes the mesh generation for Green's functions impractical.

Neural network-based learning approaches [2], being mesh-free and well-suited for high-dimensional problems, have attracted significant attention in recent years, inspiring innovations in both network architectures [24,27,30,32] and training algorithms

[15,16,25,26,33,41]. Unfortunately, a straightforward implementation via the physics-informed machine learning [3,5,6] is complicated by several issues, among which a limitation arises from the distributional nature of the Dirac delta function prohibits pointwise evaluation. To bypass this issue, existing techniques can be roughly classified into three categories: (1) replacing the Dirac delta function with a Gaussian density function [26,30], despite introducing inevitable modeling errors; (2) employing the fundamental solution to decompose the Green's function into regular and singular components [7,25,28], but the explicit expression of fundamental solutions is generally unavailable [39]; (3) reformulating the problem into a variational form, with the weak solution being learned from a minimax optimization problem [31].

In addition to the aforementioned limitations, another fundamental challenge emerges from the fact that standard neural networks usually lack the capacity to represent/capture singular functions, regardless of whether the training strategy is supervised [15,16,24] or unsupervised [26,27,33,41], or whether the training loss function is constructed in a strong or variational manner [26,27,31]. To handle the singular behavior, rational activation functions [15,32] are proposed to improve the approximation capacity of neural networks, while the low-rank [33], multiscale [30], and hierarchical [24] structures inherent in the Green's function can also be incorporated into the modification of neural network architectures.

Unlike prior learning approaches, our proposed method first embeds the Green's functions within a higher-dimensional space by introducing an augmented variable, after which a smooth neural network is adopted for the parametric approximation. Our strategy also facilitates the effective treatment of the Dirac delta function without relying on Gaussian approximation or fundamental solution.

2.2. Iterative methods with pre-trained neural operators

Traditional numerical methods for solving problem (1) typically involves discretization through finite difference or finite element methods [8,40], yielding a linear system of equations

$$AU = F \tag{4}$$

where A is the coefficient matrix, U the vector of unknowns, and F the right-hand side. Iterative solution techniques [42,43] have been widely employed to solve such systems, but often exhibit deteriorating performance under mesh refinement as the coefficient matrix becomes increasingly ill-conditioned. Preconditioning is therefore essential, that is, instead of (4) one solves

$$BAU = BF \tag{5}$$

where the preconditioner B should approximate A^{-1} effectively so as to both cluster the eigenvalues of BA and reduce its condition number [43]. Matrix factorization-based preconditioners, such as the damped Jacobi, Gauss-Seidel, and successive over-relaxation methods, are well-established but exhibit slow convergence rates in practice [44,45]. Multilevel approaches, such as the geometric multigrid [43,46] and domain decomposition [47] methods, achieve improved performance by addressing errors across different scales of spatial discretization.

In addition to classical preconditioning strategies, pre-trained neural operators [13,14] have recently emerged as a new class of preconditioners by exploiting their inherent spectral bias [48] to improve the spectral properties of the preconditioned system [20,22,49]. More precisely, neural operators, such as the deep operator network [13] and Fourier neural operator [14], are neural networks that are commonly trained in a supervised learning manner to approximate the inverse of the differential operator for Eq. (1). Owing to their mesh-independent nature, these pre-trained models permit seamless implementation at the discrete level, particularly when deployed as the preconditioner [20,22,23,49] or the coarse-grid solver [20,50]. Despite their limited overall accuracy, the spectral bias of neural operators toward low-frequency modes makes them well-suited for multilevel preconditioners.

Moreover, the spectral bias inherent in pre-trained neural operators also allows their integration with classical iterative methods, facilitating the development of hybrid schemes with enhanced convergence properties [21]. To be specific, given an initial guess $U^{[1]}$, the classical iterative solver for our discrete system (5) reads

$$U^{[k+1]} = U^{[k]} + B(f - AU^{[k]}) \tag{6}$$

where B represents a classic preconditioner that is effective for eliminating high-frequency error components [42,43] but exhibits limited efficacy for low-frequency errors (typically referred to as the smoothing effects). The other approach employs the pre-trained neural operator $\mathcal{B}$ to construct the following iterative update

$$U^{[k+1]} = U^{[k]} + \mathcal{B}(f - AU^{[k]}) \tag{7}$$

which preferentially attenuates low-frequency errors due to the spectral bias of neural operators [21–23], but at the cost of introducing additional high-frequency errors owing to their limited accuracy. As a direct consequence, a natural extension is to combine both iterative schemes into a hybrid method [21,23] that improves convergence across the entire spectrum of error modes [22]. Moreover, the hybrid method has been successfully integrated into the multigrid framework as pre- and post-smoothing procedures [51], proving valuable for hierarchical solvers.

While neural operators have demonstrated notable potential in accelerating classical iterative methods, either as preconditioners or within hybrid iterative schemes, the majority of existing implementations [21–23] rely on data-driven techniques that raise fundamental concerns regarding generalization and interpretability [4]. Besides, the challenge of acquiring high-quality and diverse training data remains a critical bottleneck in the development of robust neural operators, while the lack of transparency hinders their reliability in practical applications.

Unlike widely adopted data-driven models, our pre-trained neural operator is constructed by directly learning the Green's functions (3) within an unsupervised learning regime, thereby removing dependence on labelled data and enhancing interpretability. Accordingly, our preconditioner is explicitly expressed in a matrix representation, in contrast to the end-to-end mapping realized by the deep operator network or Fourier neural operator. Though distinct training strategies are employed, the spectral bias inherent in trained models remains unchanged, therefore enabling the acceleration of classical iterative methods through our learned Green's functions.

3. Singularity-encoded Green's functions

In this section, the learning algorithm for our singularity-encoded Green's function is presented, accompanied by an empirical analysis that reveals the spectral bias of trained neural network models. We begin by introducing an augmented variable to embed the governing equations into a one-order higher-dimensional space, demonstrating its effectiveness in resolving singularities via a benchmark problem. Next, the algorithmic framework is detailed, followed by a spectral analysis of our trained models that motivates the design of hybrid iterative solvers.

3.1. Green's functions embedded in the one-order higher dimensional space

The primary difficulty in learning Green's function stems from its inherent singular behavior near $\boldsymbol{x} = \boldsymbol{y}$, i.e.,

$$c(x)[\![\nabla G(x,y)]\!] = -1 \quad \text{for } d = 1, \quad \text{and} \quad G(\boldsymbol{x},\boldsymbol{y}) \sim \begin{cases} \ln\|\boldsymbol{x}-\boldsymbol{y}\|, & \text{for } d = 2, \\ \|\boldsymbol{x}-\boldsymbol{y}\|^{2-d}, & \text{for } d \geq 3, \end{cases} \tag{8}$$

which challenges both the approximation capacity of neural networks [32] and the practical training procedure via auto-differentiation [52]. In this work, an augmented variable $\varphi(\boldsymbol{x},\boldsymbol{y})$ is incorporated as an additional input feature into our representation of Green's function (referred to as the "singularity-encoded Green's function"):

$$G(\boldsymbol{x},\boldsymbol{y}) = \widehat{G}(\boldsymbol{x},\boldsymbol{y},\varphi(\boldsymbol{x},\boldsymbol{y})) \quad \text{with} \quad \varphi(\boldsymbol{x},\boldsymbol{y}) = \begin{cases} |x-y|, & \text{for } d = 1, \\ \ln\|\boldsymbol{x}-\boldsymbol{y}\|, & \text{for } d = 2, \\ \|\boldsymbol{x}-\boldsymbol{y}\|^{2-d}, & \text{for } d \geq 3, \end{cases} \tag{9}$$

thereby facilitating the resolution of singularities during the training of neural networks. Throughout our paper, $[\![\cdot]\!]$ denotes the difference of quantity across the discontinuity interface $\Gamma = \{x \mid x = y\}$ [39].

Before introducing the neural network parametrization of our Green's function, a direct calculation implies

$$\begin{aligned} \nabla\cdot\big(c(\boldsymbol{x})\nabla G(\boldsymbol{x},\boldsymbol{y})\big) = \; & \nabla c(\boldsymbol{x})\cdot\big(\nabla_{\boldsymbol{x}}\widehat{G}(\boldsymbol{x},\boldsymbol{y},\varphi(\boldsymbol{x},\boldsymbol{y})) + \nabla\varphi(\boldsymbol{x},\boldsymbol{y})\partial_z\widehat{G}(\boldsymbol{x},\boldsymbol{y},\varphi(\boldsymbol{x},\boldsymbol{y}))\big) \\ & + c(\boldsymbol{x})\big(\Delta\varphi(\boldsymbol{x},\boldsymbol{y})\partial_z\widehat{G}(\boldsymbol{x},\boldsymbol{y},\varphi(\boldsymbol{x},\boldsymbol{y})) + \Delta_{\boldsymbol{x}}\widehat{G}(\boldsymbol{x},\boldsymbol{y},\varphi(\boldsymbol{x},\boldsymbol{y})) \\ & + 2\nabla\varphi(\boldsymbol{x},\boldsymbol{y})\cdot\nabla_{\boldsymbol{x}}(\partial_z\widehat{G}(\boldsymbol{x},\boldsymbol{y},\varphi(\boldsymbol{x},\boldsymbol{y}))) + \|\nabla\varphi(\boldsymbol{x},\boldsymbol{y})\|^2\partial_{zz}\widehat{G}(\boldsymbol{x},\boldsymbol{y},\varphi(\boldsymbol{x},\boldsymbol{y}))\big) \end{aligned}$$

where $\nabla_{\boldsymbol{x}}$, $\Delta_{\boldsymbol{x}}$, and ∂_z represent partial derivatives with respect to the $\boldsymbol{x}$- or z-variable of function $\widehat{G}(\boldsymbol{x},\boldsymbol{y},z)$,

$$\nabla\varphi(\boldsymbol{x},\boldsymbol{y}) = \begin{cases} \operatorname{sgn}(x-y), & \text{for } d = 1, \\ \|\boldsymbol{x}-\boldsymbol{y}\|^{-2}(\boldsymbol{x}-\boldsymbol{y}), & \text{for } d = 2, \\ (2-d)\|\boldsymbol{x}-\boldsymbol{y}\|^{-d}(\boldsymbol{x}-\boldsymbol{y}), & \text{for } d \geq 3, \end{cases} \quad \text{and} \quad \Delta\varphi(\boldsymbol{x},\boldsymbol{y}) = \begin{cases} 0, & \text{for } d = 1, \\ 0, & \text{for } d = 2, \\ 0, & \text{for } d \geq 3. \end{cases}$$

To account for the δ-function in Eq. (3), we integrate over some ball $B_\epsilon(\boldsymbol{y}) = \{\boldsymbol{x} \mid \|\boldsymbol{x}-\boldsymbol{y}\| < \epsilon\}$ to obtain the so-called normalization condition for each $\boldsymbol{x} \in \Gamma = \{\boldsymbol{x} \mid \boldsymbol{x} = \boldsymbol{y} \in \Omega\}$, namely,

$$-\int_{B_\epsilon(\boldsymbol{y})} \nabla\cdot(c(\boldsymbol{x})\nabla G(\boldsymbol{x},\boldsymbol{y}))\,d\boldsymbol{x} = -\int_{\partial B_\epsilon(\boldsymbol{y})} c(\boldsymbol{x})\nabla G(\boldsymbol{x},\boldsymbol{y})\cdot\boldsymbol{n}_{\boldsymbol{x}}\,dS(\boldsymbol{x}) = 1,$$

where $\boldsymbol{n}_{\boldsymbol{x}} = \frac{\boldsymbol{x}-\boldsymbol{y}}{\|\boldsymbol{x}-\boldsymbol{y}\|}$ is the unit outer normal vector along the boundary $\partial B_\epsilon(\mathbf{y})$. Accordingly, it can be recast as

$$-\int_{\partial B_\epsilon(\boldsymbol{y})} c(\boldsymbol{x})\Big(\nabla_{\boldsymbol{x}}\widehat{G}(\boldsymbol{x},\boldsymbol{y},\varphi(\boldsymbol{x},\boldsymbol{y})) + \nabla\varphi(\boldsymbol{x},\boldsymbol{y})\partial_z\widehat{G}(\boldsymbol{x},\boldsymbol{y},\varphi(\boldsymbol{x},\boldsymbol{y}))\Big)\cdot\boldsymbol{n}_{\boldsymbol{x}}\,dS(\boldsymbol{x}) = 1 \tag{10}$$

in terms of our singularity-encoded Green's function. It is noteworthy that for $d = 1$, an equivalent expression of (10) can be obtained by sending $\epsilon \to 0$, namely,

$$-2c(x)\partial_z\widehat{G}(x,y,\varphi(x,y)) = -2c(x)\partial_z\widehat{G}(x,y,|x-y|) = 1, \quad \text{for } x \in \Gamma = \{x \mid x = y\}, \tag{11}$$

which allows the derivative jump of Green's function (8) to be accurately represented by our singularity-encoded Green's function. In other words, the piecewise-continuous Green's function $G(x,y)$ is mapped onto two disjoint hyperplanes within a 3-dimensional space (see Fig. 1 for instance), which admits a continuous extension to the entire space $G(x,y,z)$ as ensured by the Whitney extension theorem [53,54].

To sum up, the problem satisfied by our singularity-encoded Green's function $\widehat{G}(\boldsymbol{x}, \boldsymbol{y}, \varphi(\boldsymbol{x}, \boldsymbol{y}))$ is defined as

$$\begin{aligned} -\nabla c \cdot \left(\nabla_{\boldsymbol{x}} \widehat{G} + \nabla\varphi \partial_z \widehat{G}\right) - c\left(\Delta_{\boldsymbol{x}} \widehat{G} + 2\nabla\varphi \cdot \nabla_{\boldsymbol{x}}(\partial_z \widehat{G}) + \|\nabla\varphi\|^2 \partial_{zz} \widehat{G}\right) = 0, \quad & \text{for } \boldsymbol{x} \in \Omega \setminus \Gamma, \\ -\int_{\partial B_\epsilon(\boldsymbol{y})} c\left(\nabla_{\boldsymbol{x}} \widehat{G} + \nabla\varphi \partial_z \widehat{G}\right) \cdot \boldsymbol{n}_{\boldsymbol{x}} \, dS(\boldsymbol{x}) = 1, \quad & \text{for } \boldsymbol{x} \in \Gamma = \{\boldsymbol{x} \mid \boldsymbol{x} = \boldsymbol{y}\}, \\ \widehat{G} = 0, \quad & \text{for } \boldsymbol{x} \in \partial\Omega, \end{aligned} \tag{12}$$

for any fixed $\boldsymbol{y} \in \Omega$ and any $\epsilon > 0$, which is embedded within a one-order higher-dimensional space compared to its original form (3). Here, and in what follows, we employ the general form (10) for clarity, reserving the specialized one-dimensional case (11) whenever necessary.

In contrast to the derivative jump in (8), the singularity inherent to the Green's function becomes more severe in higher dimensions, posing challenges for conventional numerical methods due to the divergence of derivatives as $\boldsymbol{x} \to \boldsymbol{y}$. Physics-informed neural networks [3,5,6], which employ the automatic differentiation to calculate diverging derivatives near $\boldsymbol{x} = \boldsymbol{y}$, are no exception. Thanks to the inclusion of domain knowledge via augmenting the input variables (9), our computation of singular first-order derivatives in (10) is realized through

$$\nabla G(\boldsymbol{x}, \boldsymbol{y}) = \nabla \widehat{G}(\boldsymbol{x}, \boldsymbol{y}, \varphi(\boldsymbol{x}, \boldsymbol{y})) = \nabla_{\boldsymbol{x}} \widehat{G}(\boldsymbol{x}, \boldsymbol{y}, \varphi(\boldsymbol{x}, \boldsymbol{y})) + \nabla\varphi(\boldsymbol{x}, \boldsymbol{y}) \partial_z \widehat{G}(\boldsymbol{x}, \boldsymbol{y}, \varphi(\boldsymbol{x}, \boldsymbol{y})), \tag{13}$$

in which $\nabla\varphi(\boldsymbol{x}, \boldsymbol{y})$ grows at least at the same rate as $\nabla G(\boldsymbol{x}, \boldsymbol{y})$. As a result, satisfactory accuracy could be attained for ensuring the constraint (10) with derivatives $\nabla_{\boldsymbol{x}} \widehat{G}(\boldsymbol{x}, \boldsymbol{y}, \varphi(\boldsymbol{x}, \boldsymbol{y}))$ and $\partial_z \widehat{G}(\boldsymbol{x}, \boldsymbol{y}, \varphi(\boldsymbol{x}, \boldsymbol{y}))$ remaining bounded.

Remark 1. In fact, constructing the augmented variable for $d = 2, 3$ requires only a rough upper-bound estimation of singularities, which is sufficient to bound partial derivatives of our singularity-encoded Green's functions (13). For instance, in $d = 2$ cases where the exact Green's function exhibits a logarithmic singularity (8), our method remains effective when using $\varphi(\boldsymbol{x}, \boldsymbol{y}) = \|\boldsymbol{x} - \boldsymbol{y}\|^{-0.2}$ as numerically reported in Appendix A.

Remark 2. While the radius parameter vanishes for $d = 1$, the imposition of a flux normalization condition for $d \geq 2$ prompts an analysis bridging the true Green's function with its flux-normalized counterpart. To be specific, for each fixed point $\boldsymbol{y} \in \Omega$, we define $G(\cdot, \boldsymbol{y}) \in W^{1,1}(\Omega)$ (not relabelled) as the function satisfying

$$\begin{aligned} -\nabla \cdot (c(\boldsymbol{x}) \nabla G(\boldsymbol{x}, \boldsymbol{y})) = 0, \quad & \text{for } \boldsymbol{x} \in \Omega \setminus \{\boldsymbol{y}\}, \\ -\int_{\partial B_\epsilon(\boldsymbol{y})} c(\boldsymbol{x}) \nabla G(\boldsymbol{x}, \boldsymbol{y}) \cdot \boldsymbol{n}_x \, dS(\boldsymbol{x}) = 1, \quad & \text{for any } \epsilon > 0, \\ G(\boldsymbol{x}, \boldsymbol{y}) = 0, \quad & \text{for } \boldsymbol{x} \in \partial\Omega. \end{aligned} \tag{14}$$

Then, in the sense of distributions, it follows that

$$\begin{aligned} -\nabla \cdot (c(\boldsymbol{x}) \nabla G(\boldsymbol{x}, \boldsymbol{y})) = \delta(\boldsymbol{x} - \boldsymbol{y}), \quad & \text{for } \boldsymbol{x} \in \Omega, \\ G(\boldsymbol{x}, \boldsymbol{y}) = 0, \quad & \text{for } \boldsymbol{x} \in \partial\Omega. \end{aligned} \tag{15}$$

For the sake of brevity, the proof is moved to Appendix B. This equivalence also motivates us to numerically approximate the point source singularity by enforcing the flux normalization constraint over a sequence of decreasing radius parameters (see Section 5.2 and Section 5.4).

3.1.1. Example: Poisson's equation in one dimension

Before detailing the learning algorithms, we present a benchmark problem to visualize the proposed singularity-encoded Green's function (12) within a one-order higher-dimensional space. To be specific, we consider the following one-dimensional Poisson problem:

$$-u''(x) = f(x), \quad \text{for } x \in \Omega = (0, 1), \tag{16}$$

with homogeneous Dirichlet boundary conditions $u(0) = u(1) = 0$ as the illustrative example. Then the boundary value problem governing its exact Green's function $G(x, y)$ is characterized by

$$\begin{aligned} -\partial_{xx} G(x, y) = 0, \quad & \text{for } x \in \Omega \setminus \Gamma, \\ [\![G(x, y)]\!] = 0, \quad [\![\partial_x G(x, y)]\!] = -1, \quad & \text{for } x \in \Gamma = \{x \mid x = y\}, \\ G(x, y) = 0, \quad & \text{for } x \in \partial\Omega, \end{aligned} \tag{17}$$

for any fixed $y \in (0, 1)$, while that of singularity-encoded Green's function $\widehat{G}(x, y, \varphi(x, y))$ takes on the form

$$\begin{aligned} -(\partial_{xx} + \partial_{zz}) \widehat{G}(x, y, \varphi(x, y)) - 2\mathrm{sgn}(x - y) \partial_{xz} \widehat{G}(x, y, \varphi(x, y)) = 0, \quad & \text{for } x \in \Omega \setminus \Gamma, \\ -2\partial_z \widehat{G}(x, y, \varphi(x, y)) = 1, \quad & \text{for } x \in \Gamma = \{x \mid x = y\}, \\ \widehat{G}(x, y, \varphi(x, y)) = 0, \quad & \text{for } x \in \partial\Omega. \end{aligned} \tag{18}$$

Notably, the exact Green's function associated with problem (16) is given by

$$G(x, y) = \begin{cases} x(1 - y), & \text{for } 0 \leq x \leq y, \\ y(1 - x), & \text{for } y \leq x \leq 1. \end{cases}$$

while the constraint $0 = [\![G(x, y)]\!] = [\![\widehat{G}(x, y, \varphi(x, y))]\!]$ for $x \in \Gamma$ is naturally enforced using $\varphi(x, y) = |x - y|$.

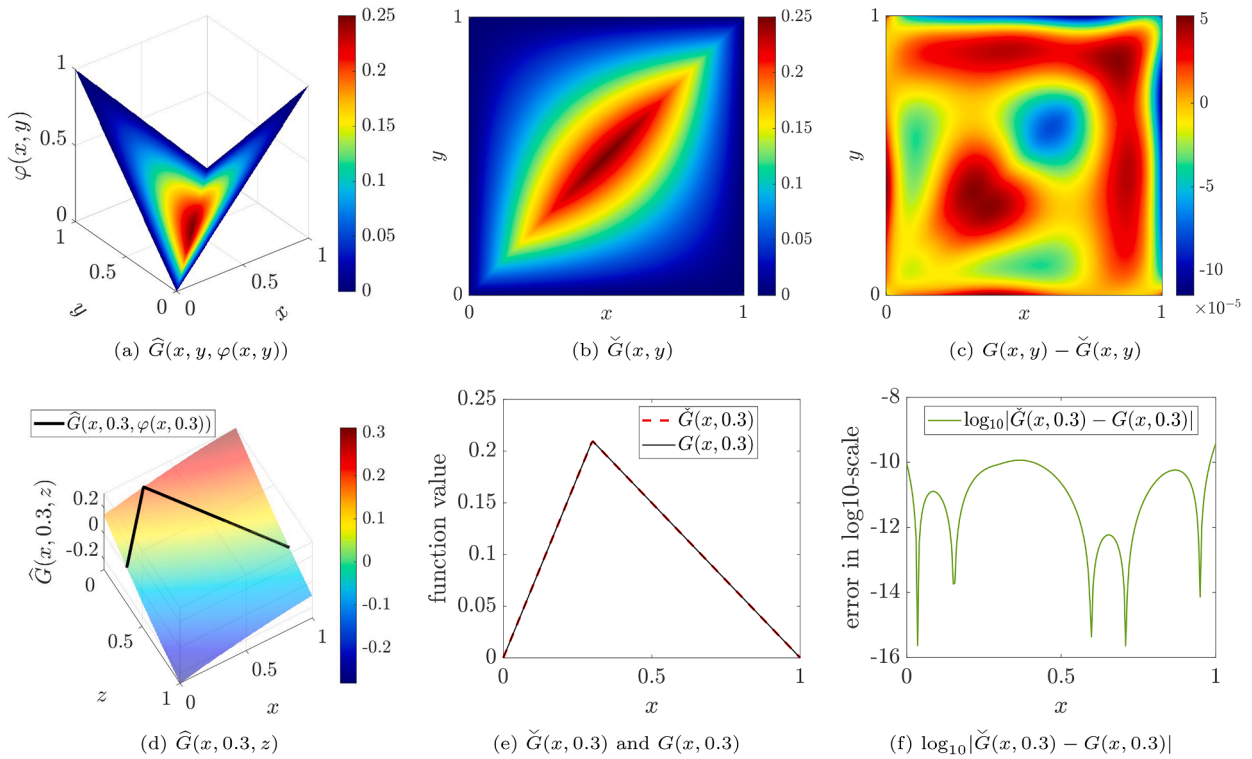


Fig. 1. Numerical singularity-encoded Green's function for the 1D Poisson Eq. (16).

Next, the unknown solution to (18) is parametrized using a fully-connected neural network [55], followed by a standard training process under the pointwise residual-minimization framework [6,56]. The trained model (still denoted as $\widehat{G}(x, y, \varphi(x, y))$ for simplicity) and its projection (denoted as $\breve{G}(x, y)$) back onto the lower-dimensional plane are depicted in Fig. 1a andb, while Fig. 1c shows its pointwise error profile compared to the true Green's function. Though the exact Green's function $G(x, 0.3)$ exhibits a jump in its derivative as depicted in Fig. 1e, its numerical counterpart $\breve{G}(x, 0.3)$ could be embedded into a smooth surface $\widehat{G}(x, 0.3, z)$ within a one-order higher-dimensional space (see Fig. 1d), thereby allowing both our trained model and numerical solution to achieve satisfactory accuracy as demonstrated in Fig. 1c and f.

3.2. Singularity-encoded neural networks

Thanks to the enhanced regularity achieved via embedding within a one-order higher-dimensional space, a fully-connected neural network [2] with smooth activation functions, e.g., $\sigma(x) = \tanh(x)$, is deployed to parametrize our singularity-encoded Green's function (not relabelled for notational simplicity), namely,

$$\widehat{G}(\boldsymbol{x}, \boldsymbol{y}, z; \theta) = (L_{D+1} \circ \sigma \circ L_D \circ \cdots \circ \sigma \circ L_1 \circ L_0)(\boldsymbol{x}, \boldsymbol{y}, z) \tag{19}$$

where $D \in \mathbb{N}_+$ denotes the number of hidden layers, L_ℓ the ℓth linear layer that applies an affine linear transformation to the incoming data, and θ the collection of all trainable parameters.

Next, our embedded problem (12) is solved under an unsupervised learning framework [6], starting with the generation of input points $Y_\Omega = \{\boldsymbol{y}_m \in \Omega\}_{m=1}^{M}$ that are sampled uniformly at random inside the physical domain. Then, for every source point $\boldsymbol{y}_m \in \Omega$, the corresponding training datasets in $\boldsymbol{x}$-variable

$$X_{\text{Reglr}} = \left\{\boldsymbol{x}_n^{\Omega}\right\}_{n=1}^{N_{\text{R}}}, \quad X_{\text{Bndry}} = \left\{\boldsymbol{x}_n^{\partial\Omega}\right\}_{n=1}^{N_{\text{B}}}, \quad \text{and} \quad X_{\text{Snglr}} = \left\{\boldsymbol{x}_n^{\partial B_\epsilon(\boldsymbol{y})} (\text{or } \boldsymbol{x}_n^{\epsilon} \text{ for brevity})\right\}_{n=1}^{N_{\text{S}}},$$

are sampled uniformly at random from the regular region $\Omega \setminus \Gamma$, boundaries $\partial\Omega$ and $\partial B_\epsilon(\boldsymbol{y})$, respectively. Here, N_{R}, N_{B}, and N_{S} indicate the batch sizes of training datasets X_{Reglr}, X_{Bndry}, and X_{Snglr}, respectively. Notably, in the one-dimensional case (11), we have $X_{\text{Snglr}} = Y_\Omega$ by sending the radius parameter $\epsilon \to 0$ in (10).

Remark 3. Note that the value of our augmented variable $z = \varphi(\boldsymbol{x}, \boldsymbol{y})$ in the singularity-encoded Green's function is uniquely determined at each collocation point $(\boldsymbol{x}, \boldsymbol{y})$, which implies that our representation of Green's function is embedded on hyperplanes within a one-order higher dimensional space rather than being globally defined (see Fig. 1a for example). As a result, the total number of input data points (or, equivalently, the execution times of forward and backward propagations) remains unchanged regardless of the increased dimensionality.

According to the residual of Eq. (12), our empirical loss functions are defined as follows

$$L_{\text{Reglr}}(\theta) = \frac{1}{M}\sum_{m=1}^{M}\left(\frac{1}{N_{\text{R}}}\sum_{n=1}^{N_{\text{R}}}\Big|\nabla c(\boldsymbol{x}_n^{\Omega})\cdot\big(\nabla_{\boldsymbol{x}}+\nabla\varphi(\boldsymbol{x}_n^{\Omega},\boldsymbol{y}_m)\partial_z\big)\widehat{G}(\boldsymbol{x}_n^{\Omega},\boldsymbol{y}_m,\varphi(\boldsymbol{x}_n^{\Omega},\boldsymbol{y}_m);\theta)+c(\boldsymbol{x}_n^{\Omega})\Big(\Delta_{\boldsymbol{x}}+2\nabla\varphi(\boldsymbol{x}_n^{\Omega},\boldsymbol{y}_m)\cdot\nabla_{\boldsymbol{x}}\,\partial_z+\|\nabla\varphi(\boldsymbol{x}_n^{\Omega},\boldsymbol{y}_m)\|^2\partial_{zz}\Big)\widehat{G}(\boldsymbol{x}_n^{\Omega},\boldsymbol{y}_m,\varphi(\boldsymbol{x}_n^{\Omega},\boldsymbol{y}_m);\theta)\Big|^2\right),$$

$$L_{\text{Snglr}}^{d\geq 2}(\theta) = \frac{1}{M}\sum_{m=1}^{M}\left(1+\frac{|\partial B_\epsilon(\boldsymbol{y}_m)|}{N_{\text{S}}}\sum_{n=1}^{N_{\text{S}}}c(\boldsymbol{x}_n^{\epsilon})\big(\nabla_{\boldsymbol{x}}+\nabla\varphi(\boldsymbol{x}_n^{\epsilon},\boldsymbol{y}_m)\partial_z\big)\widehat{G}(\boldsymbol{x}_n^{\epsilon},\boldsymbol{y}_m,\varphi(\boldsymbol{x}_n^{\epsilon},\boldsymbol{y}_m);\theta)\cdot\frac{\boldsymbol{x}_n^{\epsilon}-\boldsymbol{y}_m}{\|\boldsymbol{x}_n^{\epsilon}-\boldsymbol{y}_m\|}\right)^2,$$

$$L_{\text{Bndry}}(\theta) = \frac{1}{M}\sum_{m=1}^{M}\left(\frac{1}{N_{\text{B}}}\sum_{n=1}^{N_{\text{B}}}\Big|\widehat{G}(\boldsymbol{x}_n^{\partial\Omega},\boldsymbol{y}_m,\varphi(\boldsymbol{x}_n^{\partial\Omega},\boldsymbol{y}_m);\theta)\Big|^2\right),$$

where the normalization condition (10) induced loss function can be simplified in the one dimension, namely,

$$L_{\text{Snglr}}^{d=1}(\theta) = \frac{1}{M}\sum_{m=1}^{M}\left(\Big|2c(y_m)\partial_z\widehat{G}(y_m,y_m,\varphi(y_m,y_m);\theta)+1\Big|^2\right).$$

Moreover, in cases where the Green's function is known to be symmetric, enforcing this property through a penalty term defined for $\boldsymbol{x}\neq\boldsymbol{y}$

$$L_{\text{Symtr}}(\theta) = \frac{1}{M}\sum_{m=1}^{M}\left(\frac{1}{N_{\text{R}}}\sum_{n=1}^{N_{\text{R}}}\Big|\widehat{G}(\boldsymbol{x}_n^{\Omega},\boldsymbol{y}_m,\varphi(\boldsymbol{x}_n^{\Omega},\boldsymbol{y}_m))-\widehat{G}(\boldsymbol{y}_m,\boldsymbol{x}_n^{\Omega},\varphi(\boldsymbol{y}_m,\boldsymbol{x}_n^{\Omega}))\Big|^2\right),$$

remains mathematically consistent with the governing equations. Consequently, the learning task associated with our singularity-encoded Green's function takes on the form

$$\theta^* = \arg\min_{\theta}\ L_{\text{Reglr}}(\theta)+\beta_{\text{Snglr}}L_{\text{Snglr}}(\theta)+\beta_{\text{Bndry}}L_{\text{Bndry}}(\theta)+\mathbf{1}_{\text{Symtr}}\beta_{\text{Symtr}}L_{\text{Symtr}}(\theta),$$

where β_{Snglr}, β_{Bndry}, $\beta_{\text{Symtr}}>0$ are user-defined penalty weights and $\mathbf{1}_{\text{Symtr}}$ an indicator function enforcing the symmetry constraint in conjunction with the governing equations. Notably, the loss term $L_{\text{Snglr}}^{d\geq 2}(\theta)$ can be generalized using a sequence of diminishing radius parameters instead of a single value, given that the normalization condition (10) is imposed over a series of shrinking balls.

Upon the completion of training phase, our approximate Green's function, denoted as $\check{G}(\boldsymbol{x},\boldsymbol{y})$ throughout this paper, is obtained by projecting the trained neural network model (19) back onto the original physical domain, that is,

$$\check{G}(\boldsymbol{x},\boldsymbol{y}) = \widehat{G}(\boldsymbol{x},\boldsymbol{y},\varphi(\boldsymbol{x},\boldsymbol{y});\theta^*)+\frac{\mathbf{1}_{\text{Symtr}}}{2}\Big(\widehat{G}(\boldsymbol{y},\boldsymbol{x},\varphi(\boldsymbol{y},\boldsymbol{x});\theta^*)-\widehat{G}(\boldsymbol{x},\boldsymbol{y},\varphi(\boldsymbol{x},\boldsymbol{y});\theta^*)\Big). \tag{20}$$

The implementation details are presented in Algorithm 1, with hyperparameter configurations reported in Section 5 and at our GitHub repository (https://github.com/q1sun/SgEncd_Green_Function).

Algorithm 1 Singularity-encoded neural networks for learning Green's functions.

% *Preparation*
– generate training datasets Y_Ω, X_{Reglr}, X_{Bndry}, and X_{Snglr};
– calculate the augmented input $\varphi(\boldsymbol{x},\boldsymbol{y})$ for each sample point;
– set the indicator $\mathbf{1}_{\text{Symtr}}=1$ if the Green's function is symmetric, and 0 otherwise;
% *Training Process*
– construct and initialize the neural network model $\widehat{G}(\boldsymbol{x},\boldsymbol{y},z;\theta)$;
while the maximum number of epochs is not reached **do**
– draw mini-batch data uniformly at random from training datasets;
– network training on shuffled datasets with a suitable learning rate and penalty coefficients:

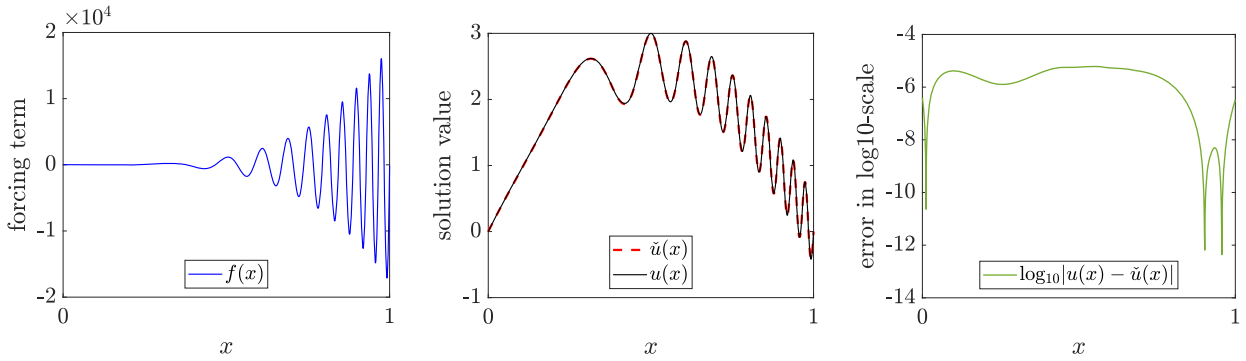
$$\theta^* = \arg\min_{\theta}\ L_{\text{Reglr}}(\theta)+\beta_{\text{Snglr}}L_{\text{Snglr}}(\theta)+\beta_{\text{Bndry}}L_{\text{Bndry}}(\theta)+\mathbf{1}_{\text{Symtr}}\beta_{\text{Symtr}}L_{\text{Symtr}}(\theta);$$

end while
% *Testing Process*
– forward pass of the trained neural network model:

$$\check{G}(\boldsymbol{x},\boldsymbol{y}) = \widehat{G}(\boldsymbol{x},\boldsymbol{y},\varphi(\boldsymbol{x},\boldsymbol{y});\theta^*)+\frac{\mathbf{1}_{\text{Symtr}}}{2}\Big(\widehat{G}(\boldsymbol{y},\boldsymbol{x},\varphi(\boldsymbol{y},\boldsymbol{x});\theta^*)-\widehat{G}(\boldsymbol{x},\boldsymbol{y},\varphi(\boldsymbol{x},\boldsymbol{y});\theta^*)\Big).$$

As a direct result, our approximate Green's function (20) can be used for computing the solution of problem (1) for arbitrary forcing terms, namely, through the numerical integration of (2)

$$u(\boldsymbol{x})\approx\check{u}(\boldsymbol{x}) = \sum_{T_p\in\mathcal{T}_h}\sum_{(\omega_y,\boldsymbol{y}_q)}\omega_y f(\boldsymbol{y}_q)\check{G}(\boldsymbol{x},\boldsymbol{y}_q), \tag{21}$$



(a) $f(x)$ (b) $u(x)$ and $\check{u}(x)$ (c) $\log_{10}|u(x) - \check{u}(x)|$

Fig. 2. Numerical validation via fast solver for the 1D Poisson Eq. (16).

in which $\mathcal{T}_h$ indicates a triangulation of domain Ω with the mesh size $h > 0$, and $(\omega_y, \boldsymbol{y}_q)$ the quadrature weights and points associated with the pth element $T_p \in \mathcal{T}_h$. Notably, (21) only requires a forward pass of our trained neural network model, enabling real-time efficiency in the numerical solution of boundary value problem (1). For instance, we choose the forcing term $f(x) = 20 - 60\pi x\cos(20\pi x^3) + 1800\pi^2 x^4 \sin(20\pi x^3)$ for our model problem (16), where the exact solution is given by $u(x) = 10x - 10x^2 + 0.5\sin(20\pi x^3)$. The computational results shown in Fig. 2 demonstrate that our approach (21) achieves good approximation accuracy without requiring the numerical discretization of the differential Eq. (16).

3.3. Spectral bias of singularity-encoded Green's functions

Despite its computational efficiency as a fast solver (21) for elliptic boundary value problems with diverse source terms, the spectral bias [57,58] of our singularity-encoded Green's functions (20) is addressed in this section, which motivates the design of hybrid iterative method in the next section. To begin with, we revisit the Mercer's theorem [59] for representing measurable kernels, which is also known as the Karhunen-Loéve expansion [60].

Lemma 1. *Suppose that the kernel function $K(\boldsymbol{x}, \boldsymbol{y}) \in L^2(\Omega \times \Omega)$ is symmetric and non-negative definite, then its corresponding integral operator*

$$\mathcal{K} : L^2(\Omega) \to L^2(\Omega), \quad (\mathcal{K}\phi)(\boldsymbol{x}) := \int_\Omega K(\boldsymbol{x}, \boldsymbol{y})\phi(\boldsymbol{y})\, d\boldsymbol{y} \quad \text{for any } \phi \in L^2(\Omega)$$

is continuous, self-adjoint and Hilbert-Schmidt, admitting a countable sequence $\{\mu_j, \phi_j(\boldsymbol{x})\}_{j\geq 1}$ of eigenpairs with $\mathbb{R} \ni \mu_j \searrow 0$ as $j \to \infty$. Moreover, the kernel $K(\boldsymbol{x}, \boldsymbol{y})$ can be represented in the series expansion

$$K(\boldsymbol{x}, \boldsymbol{y}) = \sum_{j=1}^{\infty} \mu_j \phi_j(\boldsymbol{x})\phi_j(\boldsymbol{y}) \tag{22}$$

that converges in the L^2 norm [61], namely, the truncation of expansion (22) after first $N \in \mathbb{N}_+$ terms (indicated by $K_N(\boldsymbol{x}, \boldsymbol{y})$ in what follows) satisfies the optimality condition

$$\|K(\boldsymbol{x}, \boldsymbol{y}) - K_N(\boldsymbol{x}, \boldsymbol{y})\|^2_{L^2(\Omega\times\Omega)} = \left\|K(\boldsymbol{x}, \boldsymbol{y}) - \sum_{j=1}^{N} \mu_j \phi_j(\boldsymbol{x})\phi_j(\boldsymbol{y})\right\|^2_{L^2(\Omega\times\Omega)} = \sum_{j=N+1}^{\infty} \mu_j^2.$$

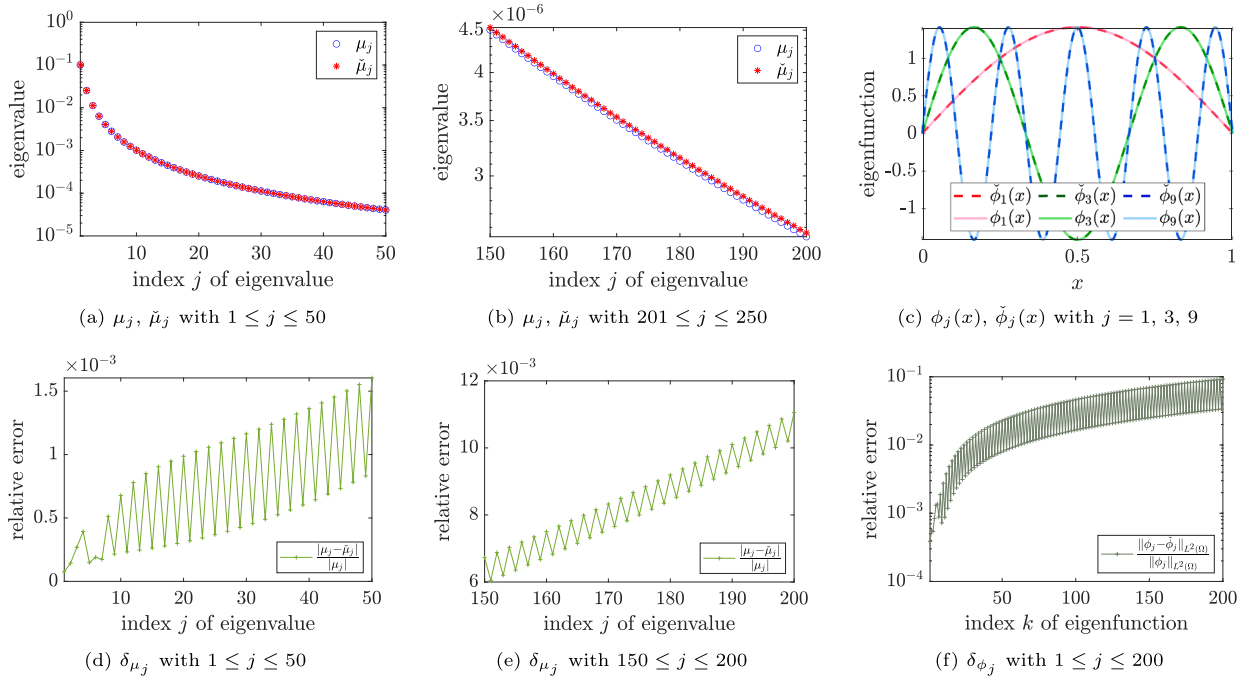
Recall that for dimensions $1 \leq d \leq 3$, our singularity-encoded Green's function is symmetric by construction (20) and L^2-integrable due to our inclusion of weakly singular augmented variables (9). Then, by Lemma 1, the eigenvalue problem admits a well-posed variational form: find $0 \neq \mu_j \in \mathbb{R}$ and $0 \neq \phi_j(\boldsymbol{x}) \in L^2(\Omega)$ such that

$$\int_\Omega \int_\Omega \check{G}(\boldsymbol{x}, \boldsymbol{y})\phi_j(\boldsymbol{y})\psi(\boldsymbol{x})\, d\boldsymbol{y}d\boldsymbol{x} = \mu_j \int_\Omega \phi_j(\boldsymbol{x})\psi(\boldsymbol{x})\, d\boldsymbol{x} \quad \text{for any } \psi(\boldsymbol{x}) \in L^2(\Omega).$$

This enables spectral analysis of our singularity-encoded Green's function through the Galerkin approximation: find $0 \neq \check{\mu}_j \in \mathbb{R}$ and $\check{\phi}_j(\boldsymbol{x}) \in V_h \subset L^2(\Omega)$ such that

$$\int_\Omega \int_\Omega \check{G}(\boldsymbol{x}, \boldsymbol{y})\check{\phi}_j(\boldsymbol{y})\psi_h(\boldsymbol{x})\, d\boldsymbol{y}d\boldsymbol{x} = \check{\mu}_j \int_\Omega \check{\phi}_j(\boldsymbol{x})\psi_h(\boldsymbol{x})\, d\boldsymbol{x} \quad \text{for any } \psi_h(\boldsymbol{x}) \in V_h, \tag{23}$$

where V_h denotes the finite element space, e.g., the space of linear or quadratic Lagrange elements. Here, the singular value $\check{G}(\boldsymbol{x}, \boldsymbol{x})$ is approximated by averaging over some balls centered at $\boldsymbol{x} \in \Omega$, in accordance with the Lebesgue differentiation theorem [62] for the local integrable Green's functions ($d = 2$ and 3).



(a) μ_j, $\check{\mu}_j$ with $1 \le j \le 50$ (b) μ_j, $\check{\mu}_j$ with $201 \le j \le 250$ (c) $\phi_j(x)$, $\check{\phi}_j(x)$ with $j = 1, 3, 9$

(d) δ_{μ_j} with $1 \le j \le 50$ (e) δ_{μ_j} with $150 \le j \le 200$ (f) δ_{ϕ_j} with $1 \le j \le 200$

Fig. 3. Numerical validation via spectral comparison for the 1D Poisson Eq. (16).

The spectral properties of the singularity-encoded Green's function, as characterized numerically through the eigenvalue problem (23) of our illustrative example (16) in Fig. 3a, exhibit notable consistency across variable-coefficients and higher-dimensional settings (see Section 5). Specifically, we adopt quadratic elements with regular mesh width $h = 2^{-10}$ for solving the eigenvalue problem (23), with relative errors being defined as

$$\delta_\mu = \frac{|\check{\mu}_j - \mu_j|}{|\mu_j|} \quad \text{and} \quad \delta_\phi = \frac{\|\check{\phi}_j(x) - \phi_j(x)\|_{L^2(\Omega)}}{\|\phi_j(x)\|_{L^2(\Omega)}}.$$

As evidenced in Fig. 3d, e, and f, our singularity-encoded Green's function exhibits markedly better accuracy in the low-frequency regimes compared to its high-frequency counterparts, which aligns with the spectral bias [57] (or frequency principle [48]) observed and studied across diverse applications.

While the spectral bias inherent in our singularity-encoded Green's function might appear as a deficiency, we show in Section 4.2 that it can be utilized to accelerate the convergence of classical iterative methods, particularly for low-frequency components. In fact, the presence of spectral bias persists in the deep operator network [13] or the Fourier neural operator [14], motivating the design of efficient numerical methods through hybrid iterative schemes [21] or preconditioning strategies [20].

4. Iterative methods enhanced by singularity-encoded Green's functions

In this section, we focus on the discrete system of elliptic problem (1), where the pre-trained singularity-encoded Green's function is utilized to improve the convergence speed of iterative solvers, either by constructing effective preconditioners in Section 4.1 or designing efficient hybrid iterative schemes in Section 4.2. On the one hand, our trained surrogate model inspires the development of both dense preconditioning matrices for small-scale systems and the two-level overlapping additive Schwarz preconditioners for large-scale linear systems. On the other hand, hybrid iterative strategies are presented by leveraging the spectral bias of trained models, followed by integrating into the geometric multigrid framework to accelerate convergence. The effectiveness of our proposed methods is validated via the one-dimensional example (18) for illustration, while more examples are presented in Section 5.

4.1. Preconditioned system via singularity-encoded Green's functions

Recall that the exact Green's function serves as the inverse of a differential operator in the continuous setting (2). Discretizing the elliptic boundary value problem (1) via finite difference or finite element methods [8,40] on a domain triangulation $\mathcal{T}_h$ with mesh size $h > 0$ gives rise to a linear algebraic system

$$AU = F \tag{24}$$

where $A \in \mathbb{R}^{n\times n}$ denotes a symmetric positive definite matrix, $F \in \mathbb{R}^n$ the right-hand side, and $U \in \mathbb{R}^n$ the vector of unknowns. To improve the convergence of iterative solvers, it is common to consider a preconditioned system

$$BAU = BF$$

where a preconditioner $B \in \mathbb{R}^{n\times n} \approx A^{-1}$ is incorporated to improve the spectral properties of (24). Note that A^{-1} corresponds to a discrete counterpart of the exact Green's function [8], hence a promising preconditioner can be constructed by evaluating our singularity-encoded Green's functions (20) at the grid points $\{\boldsymbol{x}_i\}_{i=1}^n$

$$(\check{B})_{ij} = \check{G}(\boldsymbol{x}_i, \boldsymbol{x}_j), \quad \text{for } 1 \le i, j \le n. \tag{25}$$

Though the construction of preconditioner (25) remains valid under mesh refinement, the global support property of the Green's function (3) would result in a dense matrix [63] that may hinder computational efficiency [42]. As such, the preconditioning matrix (25) may not be suitable for solving large-scale systems, as both the computational cost and the accumulation of round-off errors become difficulty to manage.

To address large-scale linear systems (24), we consider an additive preconditioner (not relabelled for simplicity) within the framework of two-level overlapping Schwarz methods [20,47]. To be specific, let $\{\Omega_\ell\}_{\ell=1}^L$ denote an overlapping decomposition of the computational domain and let R_ℓ be the matrix representation of restriction operator on the subdomain Ω_ℓ [20,47], the local matrix associated with Ω_ℓ can then be formulated as

$$A_\ell = R_\ell A R_\ell^T$$

for $1 \le \ell \le L$. On the other hand, given a shape-regular coarse mesh $\mathcal{T}_H$ on the domain, the restriction matrix R_0 for its corresponding coarse space can be defined analogously [43,47], as well as the coarse matrix $A_0 = R_0 A R_0^T$. It is also noteworthy that the coarse matrix can be built by employing discretization methods on the coarse mesh. Consequently, the two-level overlapping additive Schwarz preconditioner [47] takes on the form

$$B = R_0^T A_0^{-1} R_0 + \sum_{\ell=1}^{L} R_\ell^T A_\ell^{-1} R_\ell, \tag{26}$$

in which the coarse solver A_0^{-1} provides a global correction mechanism that complements local solvers $\{A_\ell^{-1}\}_{\ell=1}^L$ by handling low-frequency errors that extend across multiple subdomains.

Particularly, our coarse solver is constructed by applying the same routine (25) but for the coarse grid $\{\boldsymbol{x}_i'\}_{i=1}^{n'}$ of triangulation $\mathcal{T}_H$, namely, $(\check{B}_0)_{ij} = \check{G}(\boldsymbol{x}_i', \boldsymbol{x}_j')$ for $1 \le i, j \le n'$, giving rise to an additive preconditioner of the form

$$\check{B} = R_0^T \check{B}_0 R_0 + \sum_{\ell=1}^{L} R_\ell^T A_\ell^{-1} R_\ell, \tag{27}$$

which mitigates both computational complexity and storage requirements when handling large-scale systems.

For the ease of illustration, we refer to (25) and (27) as the neural preconditioner [22] throughout this work, as they are not only established through a forward pass of the trained neural network model (20) but also inherit the spectral bias of our singularity-encoded Green's functions. Finally, the preconditioned system

$$\check{B}AU = \check{B}F \tag{28}$$

is solved using the biconjugate gradient method [42], with implementation details provided in Algorithm 2.

4.1.1. Example: Poisson's equation in one dimension

To validate the effectiveness of our neural preconditioner (25) and (27), we study in this section the linear system arising from the discretization of our illustrative example (16) on a uniform grid $\{x_i = ih\}_{i=0}^{n+1}$ with the mesh size $h = \frac{1}{n+1}$ [8]. For instance, when using linear finite elements [40], the discrete system (24) takes on the form

$$A = \frac{1}{h}\begin{bmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 & -1 \\ & & & -1 & 2 \end{bmatrix}, \quad U = \begin{bmatrix} U_1 \\ U_2 \\ \vdots \\ U_{n-1} \\ U_n \end{bmatrix}, \quad F = h\begin{bmatrix} F_1 \\ F_2 \\ \vdots \\ F_{n-1} \\ F_n \end{bmatrix}, \tag{29}$$

where $F_i = \int_0^1 f(x)\phi_i(x)dx$ for $1 \le i \le n$. To mitigate the ill-conditioning of the stiffness matrix in (29), a variety of preconditioning strategies have been developed in the literature. It is also noteworthy that for this example, the inverse of the stiffness matrix coincides with the evaluation of the exact Green's function at grid points [8].

Here, we employ the singularity-encoded Green's function pre-trained in Section 3.1.1 to construct our neural preconditioner for linear system (29). Spectral properties of our preconditioned matrix $\check{B}A$ are quantified by the ratio of its largest to smallest

Algorithm 2 Preconditioned system via singularity-encoded Green's functions.

1: % *Preparation*
2: – discretize problem (1) over the triangulation $\mathcal{T}_h$ to obtain a discrete linear system $AU = F$;
3: – employ Algorithm 1 to learn the singularity-encoded Green's function for problem (1);
4: % *Neural Preconditioner*
5: **if** small-scale linear systems **then**
6: – forward pass of the trained model with input samples $\{\boldsymbol{x}_i\}_{i=1}^n$ taken from the fine mesh $\mathcal{T}_h$:
7: $$(\breve{B})_{ij} = \breve{G}(\boldsymbol{x}_i, \boldsymbol{x}_j), \quad \text{for } 1 \le i, j \le n;$$
8: **else if** large-scale linear systems **then**
9: – start with a coarse mesh $\mathcal{T}_H$, triangulate the domain Ω into overlapping subdomains $\{\Omega_\ell\}_{\ell=1}^L$;
10: – assemble local coefficient matrices $\{A_\ell\}_{i=1}^L$ and their associated restriction matrices $\{R_\ell\}_{\ell=0}^L$;
11: – forward pass of the trained model with input samples $\{\boldsymbol{x}'_i\}_{i=1}^{n'}$ taken from the coarse mesh $\mathcal{T}_H$:
12: $$\breve{B} = R_0^T \breve{B}_0 R_0 + \sum_{\ell=1}^{L} R_\ell^T A_\ell^{-1} R_\ell \quad \text{with} \quad (\breve{B}_0)_{ij} = \breve{G}(\boldsymbol{x}'_i, \boldsymbol{x}'_j) \text{ for } 1 \le i, j \le n';$$
13: **end if**
14: % *Preconditioned System*
15: – employ the biconjugate gradient method to solve the preconditioned linear system:
16: $$\breve{B}AU = \breve{B}F.$$

Table 1
BiCG methods with and without neural preconditioning for solving the discrete 1D Poisson Eq. (16).

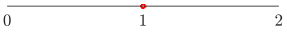
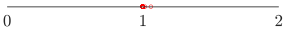
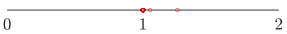
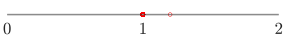
mesh size	eigenvalues of preconditioned matrix	condition number	# iterations	ℓ_2-norm error
$h = 2^{-6}$		$\kappa(\breve{B}A) = 1.01$ $\kappa(A) = 1.66 \times 10^3$	6 (63)	6.54×10^{-15} (8.78×10^{-15})
$h = 2^{-8}$		$\kappa(\breve{B}A) = 1.06$ $\kappa(A) = 2.66 \times 10^4$	6 (255)	3.27×10^{-14} (1.63×10^{-14})
$h = 2^{-10}$		$\kappa(\breve{B}A) = 1.26$ $\kappa(A) = 4.25 \times 10^5$	6 (1023)	7.60×10^{-13} (6.91×10^{-13})
$h = 2^{-12}$		$\kappa(\breve{B}A) = 2.04$ $\kappa(A) = 6.80 \times 10^6$	6 (4095)	4.89×10^{-12} (4.82×10^{-12})

Table 2
BiCG method with two-level additive Schwarz preconditioner (27) for solving the discrete 1D Poisson Eq. (16).

mesh size	coarse grid	overlap	condition number	# iterations	ℓ_2-norm error
$h = 2^{-16}$	$H = 2^{-10}$	$\delta = 30h$	$\kappa(\breve{B}A) = 7.26$	23	7.73×10^{-10}
$h = 2^{-18}$	$H = 2^{-10}$	$\delta = 30h$	$\kappa(\breve{B}A) = 5.51$	23	2.5×10^{-8}

eigenvalues, namely,

$$\kappa(\breve{B}A) = ||\breve{B}A||_2 ||(\breve{B}A)^{-1}||_2 = \frac{\lambda_{\max}}{\lambda_{\min}},$$

which reveals the clustering behavior of eigenvalues that governs the convergence speed of iterative methods.

For small-scale systems, our neural preconditioner is constructed directly through formula (25), with Table 1 comparing the performance of the biconjugate gradient method with and without preconditioning. Obviously, the condition number of coefficient matrix in problem (29) scales as $\kappa(A) = \mathcal{O}(h^2)$, while that of our preconditioned matrix is close to $\kappa(\breve{B}A) \approx \mathcal{O}(1)$ regardless of the mesh refinement. As a result, the number of iterations required for convergence becomes mesh-independent after preconditioning, in contrast to the original problems that grows as the mesh gets refined (see the last two columns in Table 1, where parentheses denotes results obtained without preconditioning). Our iteration for preconditioned system is terminated when the error, defined by the ℓ_2-norm of the difference between the iterative and exact solutions, matches that of the vanilla biconjugate gradient method.

For large-scale systems, the use of a dense preconditioning matrix (25) would render both computational cost and memory demands prohibitive, along with a notable accumulation of round-off errors. Therefore, the additive Schwarz preconditioner (27) is deployed, in which the evaluation of singularity-encoded Green's function (20) is restricted on the coarse grid to obtain $\breve{B}_0$ while the local problems $\{A_\ell^{-1}\}_{\ell=1}^L$ are of small size and directly solved. Numerical results are displayed in Table 2, which demonstrate the effectiveness of our neural preconditioner (27) as the mesh gets further refined (the unpreconditioned counterpart is omitted as it becomes impractical to solve).

4.2. Hybrid iterative methods with classical and neural preconditioners

Motivated by the spectral bias/frequency principle of singularity-encoded Green's functions (see Section 3.3), we construct hybrid strategies that enhance classical iterative methods by effectively attenuating their low-frequency errors. More specifically, our hybrid iterative methods are formulated in a residual-correction form [22,23]:

$$U^{[k+1]} = \begin{cases} U^{[k]} + \check{B}(F - AU^{[k]}), & \text{if } k \equiv 0 \ (\mathrm{mod}\ K), \\ U^{[k]} + B(F - AU^{[k]}), & \text{otherwise}, \end{cases} \tag{30}$$

where $k \in \mathbb{N}_+$ is the iteration index, $B \in \mathbb{R}^{n\times n}$ the conventional preconditioner [42] designed to approximate A^{-1} and $\check{B} \in \mathbb{R}^{n\times n}$ the preconditioning matrix deduced from our singularity-encoded Green's function (25). Here, the parameter K serves as the switching period, specifying how often the iteration alternates between the two update schemes presented in (30).

As a direct consequence of its construction (25), the neural preconditioner $\check{B}$ used in (30) inherits the spectral bias of our pre-trained singularity-encoded Green's function, which is promising in reducing low-frequency error components compared to its high-frequency counterparts. Then, our neural preconditioner is periodically applied to attenuate low-frequency errors that are difficult for conventional preconditioners to eliminate [21,22], while the high-frequency error introduced by our neural preconditioner can be rapidly reduced through classical iterations.

Accordingly, the error propagation associated with our hybrid iterative method (30) can be expressed as

$$U - U^{[\ell K]} = (I - BA)^{K-1}(I - \check{B}A)(U - U^{[(\ell-1)K]}) = \left[(I - BA)^{K-1}(I - \check{B}A)\right]^{\ell}(U - U^{[0]})$$

for any $\ell \in \mathbb{N}$, which decays to zero if and only if the amplification matrix satisfies

$$\rho\left((I - BA)^{K-1}(I - \check{B}A)\right) < 1,$$

where $\rho(\cdot)$ represents the spectral radius [23,40]. Therefore, a formal convergence analysis of our hybrid iterative scheme (30) can be established, while implementation details are summarized in Algorithm 3.

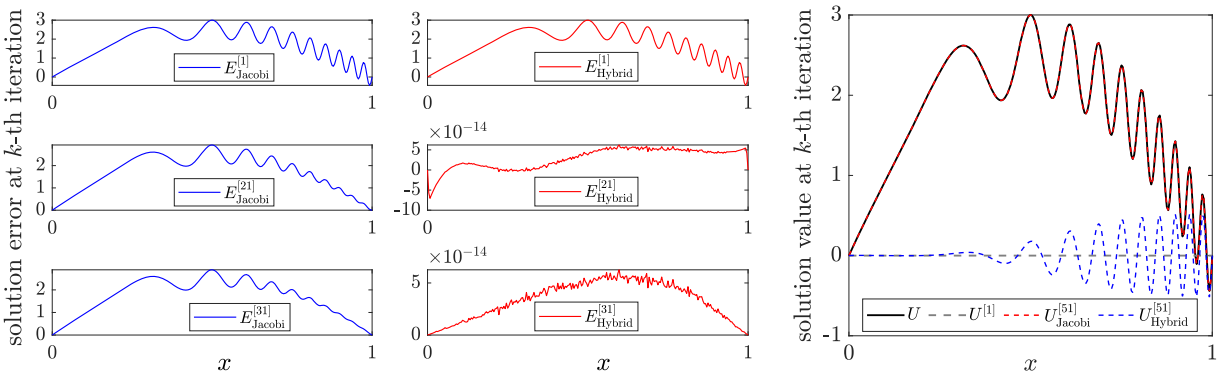
Algorithm 3 Hybrid Iterative Methods with Classical and Neural Preconditioners.

```
% Preparation
– discretize problem (1) over the triangulation T_h to obtain a discrete linear system AU = F;
– employ Algorithm 1 to learn the singularity-encoded Green's function for problem (1);
– construct the preconditioner (25) or (27) via a forward pass of the trained neural network;
% Iterative Procedure
– set the initial guess U^[1] ∈ R^n to a vector of zeros;
for k = 1, 2, 3, … do
  while U^[k] does not satisfy the stopping criteria do
    if k ≡ 0 (mod K = 2) then
      U^[k+1] = U^[k] + B̌(F − AU^[k])  {reduce low-frequency errors by our neural preconditioner}
    else
      U^[k+1] = U^[k] + B(F − AU^[k])  {reduce high-frequency errors by classical preconditioners}
    end if
  end while
end for
```

Lemma 2. [23] *There exists a correction period $K \in \mathbb{N}_+$ large enough such that the hybrid iterative method (30) converges, if the applied original iterative method converges, i.e., $\rho(I - BA) < 1$.*

Notably, the switching or correction period K is assigned relatively large values in existing studies [21–23], which seems to be consistent with the convergence result established in Lemma 2. However, low-frequency error components are not addressed until $K - 1$ iterations of the traditional method have been completed in each cycle, by which time the solution has already been smoothed by its preceding updates. To alleviate this inefficiency, we propose adopting $K = 2$ in our hybrid iterative method (30), in which the classical and neural preconditioners are applied alternatively in a sequential manner. The effectiveness of our statement is further validated via numerical experiments in Section 4.2.1 and Section 5, where $K = 2$ shows superior acceleration compared to other values.

In addition to accelerating basic iterative schemes such as the damped Jacobi method [42] (see Section 4.2.1), our hybrid approach (30) can also be integrated into the well-established geometric multigrid framework [46]. To be specific, geometric multigrid methods recursively deploy fine-grid relaxation (e.g., damped Jacobi method) to attenuate high-frequency errors with coarse-grid correction to eliminate low-frequency errors. Although efficient on structured grids, its direct extension to unstructured meshes and anisotropic problems is often hindered by the complexity of generating hierarchical meshes [21,23]. Alternatively, by substituting the pre- and post-smoothing procedures with our hybrid iterative approach (30), comparable accuracy can be achieved with a reduced number of grid levels, thereby avoiding the construction of nested hierarchical meshes (see Section 4.2.1 and Section 5).



(a) comparison of smoothing effects between methods (31) and (32) (b) comparison of iterative solutions

Fig. 4. Comparison of low-frequency error elimination for the discrete system of the 1D Poisson Eq. (16).

4.2.1. *Example: Poisson's equation in one dimension*

To demonstrate the efficacy our proposed hybrid iterative scheme (30), we consider its application to the discrete linear system (29) arising from the discretization of the illustrative example (16) and compare results with that of the damped Jacobi method [42]. More specifically, we employ the frequently used preconditioner

$$B = \omega D^{-1}$$

in which $\omega = \frac{1}{2}$ denotes a relaxation parameter [42] and D the diagonal part of our stiffness matrix A using mesh size $h = 2^{-8}$, to generate iterative solutions of the classical damped Jacobi method, namely,

$$U^{[k+1]}_{\text{Jacobi}} = U^{[k]}_{\text{Jacobi}} + \omega D^{-1}(F - AU^{[k]}_{\text{Jacobi}}). \tag{31}$$

On the other hand, the same preconditioner is utilized in a hybrid manner within our iterative scheme (3)

$$U^{[k+1]}_{\text{Hybrid}} = \begin{cases} U^{[k]}_{\text{Hybrid}} + \check{B}(F - AU^{[k]}_{\text{Hybrid}}), & \text{if } k \text{ is even,} \\ U^{[k]}_{\text{Hybrid}} + \omega D^{-1}(F - AU^{[k]}_{\text{Hybrid}}), & \text{if } k \text{ is odd.} \end{cases} \tag{32}$$

By defining the solution error, residual error, and mode-wise error at the kth iteration as

$$E^{[k]} = U - U^{[k]}, \quad R^{[k]} = F - AU^{[k]}, \quad \text{and} \quad M^{[k],j} = |\langle E^{[k]}, \xi_j \rangle|,$$

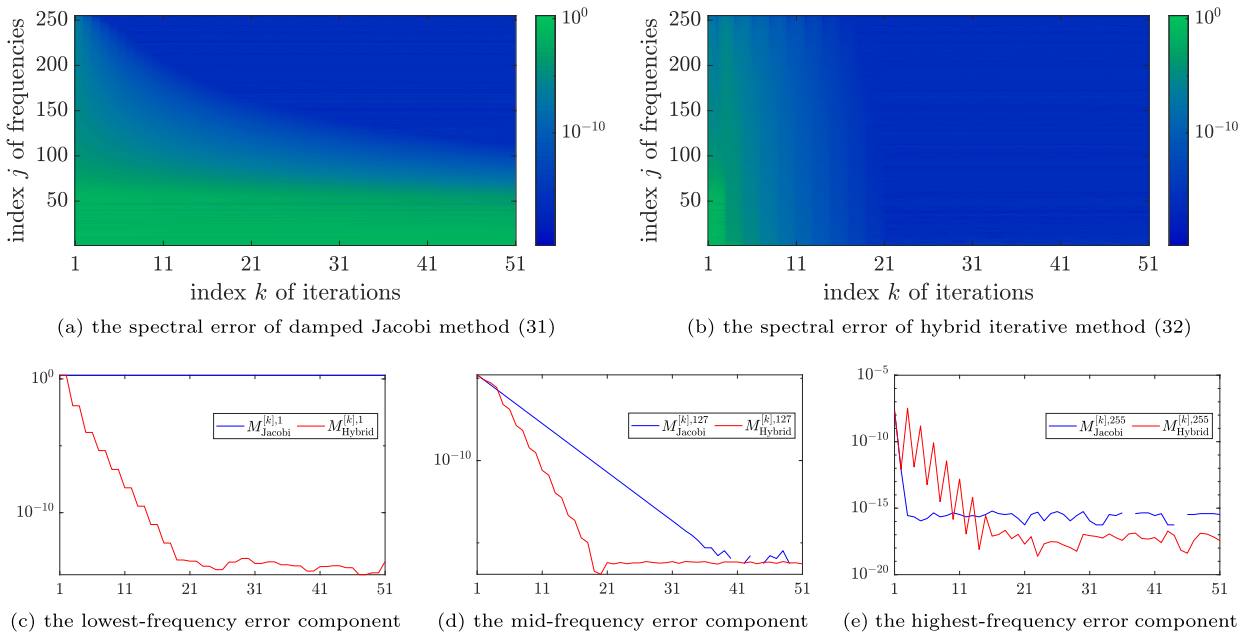
where ξ_j denotes the jth eigenvector of the amplification matrix $I - BA$, we now present a quantitative comparison of (31) and (32). As shown in Fig. 4a, the solution error computed by the damped Jacobi method (31) becomes smooth within the first few iterations, which is known as the smoothing effects whereby high-frequency error components are rapidly eliminated while its low-frequency counterparts persist. Conversely, the solution error of our hybrid iterative scheme (32) retains oscillatory behavior as displayed in Fig. 4a, owing to the extra high-frequency error introduced by our neural preconditioner. As a direct result, superior alignment with the true solution is observed in the early iterations, as demonstrated in Fig. 4b.

Moreover, Fig. 5a and b depict the mode-wise error $M^{[k],j}$ across all frequencies for both methods, while the lowest-, intermediate-, and highest-frequency error components are individually displayed in Fig. 5c, d, and e, respectively. Though the inclusion of neural preconditioner introduces additional high- frequency errors (see Fig. 5e for instance) as a consequence of its spectral bias, they can be suppressed by our subsequent execution of Jacobi method. After approximately 20 updates, our proposed approach almost achieves machine accuracy, whereas the traditional method still exhibits notable low-frequency errors.

In addition, Fig. 6a and b show the solution error and the residual error for both methods, in which different values of the switching period are utilized in Algorithm 3. The most rapid decay is attained with $K = 2$, wherein the traditional and neural preconditioners are applied alternatively during the iteration. In contrast, using a relatively larger value of K, as is common in the existing literature [21–23], impedes the prompt reduction of low-frequency errors and thereby degrades the speed of convergence.

In addition to accelerating simple iterative methods, our hybrid iterative scheme (30) could also be integrated into the multigrid framework [42], as discussed at the end of Section 4.2. Specifically, we consider the numerical solution of linear system (29) using a mesh size of $h = 2^{-8}$, while the coarsest mesh size is set to $h = 2^{-4}$ so that a direct solver can be applied. Our comparative study begins with a standard two-grid multigrid approach, whose V-cycle structure is visualized in Fig. 7a (left). Unfortunately, such a notable coarser gird may fail to represent low-frequency errors inherited from the fine grid, resulting in performance deterioration as depicted in Fig. 7b [46]. Therefore, a geometric multigrid approach with a hierarchy of five nested grids (see Fig. 7a) is employed instead, achieving satisfactory accuracy within 16 V-cycles as shown in Fig. 7b.

While uniformly refined nested meshes are ideal for multigrid solvers, their construction becomes non-trivial when applied to problems with anisotropic coefficients or complex geometries [21,40]. Thanks to the capability of our neural preconditioner in eliminating low-frequency errors, Fig. 7a implies that comparable accuracy could be achieved within a two-grid framework by employing our hybrid iterative method (30) ($K = 2$) during the pre- and post-smoothing procedures (see Fig. 7b). Besides, our hybrid



(a) the spectral error of damped Jacobi method (31)

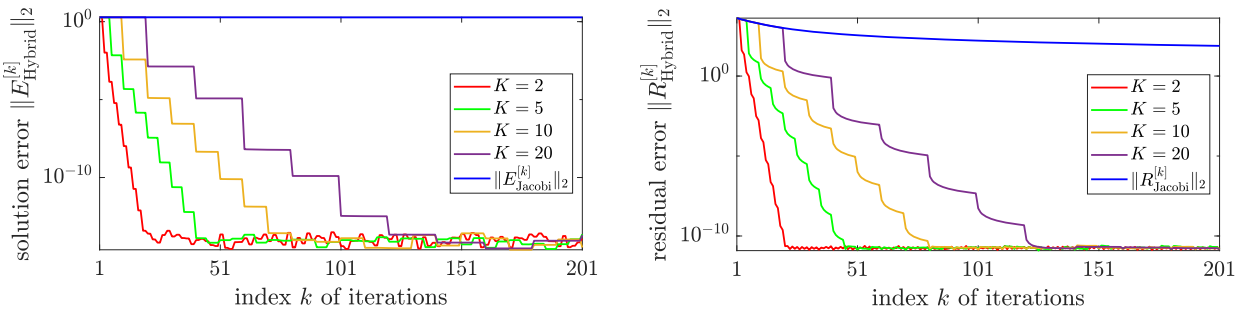
(b) the spectral error of hybrid iterative method (32)

(c) the lowest-frequency error component

(d) the mid-frequency error component

(e) the highest-frequency error component

Fig. 5. Comparison of spectral errors in iterative solvers for the discrete 1D Poisson Eq. (16).



(a) solution error using different switching periods

(b) residual error using different switching periods

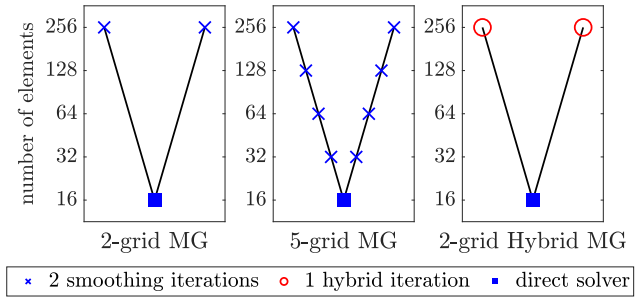
Fig. 6. Hybrid iteration (32) with different switching periods for the discrete 1D Poisson Eq. (16).

strategy enables convergence to the machine precision within 6 updates of V-cycles, demonstrating accelerated convergence while reducing its dependence on deeply nested grid hierarchies.

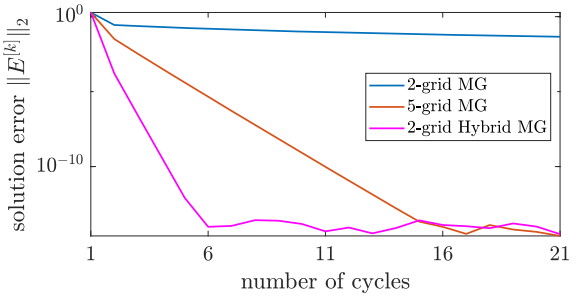
5. Numerical experiments

Beyond our illustrative example discussed in Section 3.1.1, 4.1.1, and 4.2.1, this section reports numerical studies on additional boundary value problems to demonstrate the effectiveness of our proposed methods. To be specific, each experiment starts with the learning of the true Green's function via Algorithm 1, followed by its application to accelerating conventional iterative methods, either by serving as a preconditioning matrix in Algorithm 2 or as a component of our hybrid iterative method in Algorithm 3. Particularly, the indefinite Helmholtz problem is also considered, as it poses greater challenges for classical iterative methods compared to the positive definite cases, together with problems involving nonsymmetric elliptic operators, variable coefficients, and complex domain geometries.

The introduction of an augmented variable (9), coupled with the embedding of the Green's function (3) into a one-order higher-dimensional space, facilitates the effective representation of singularity during training. Though our singularity-encoded Green's function (9) manifests increased dimensionality, the curse of dimensionality can be mitigated through the parametrization using fully-connected neural networks. Besides, the generation of input collocation points remains unaffected within the higher-dimensional space, as their associated augmented variables can be directly computed through formula (9). It is also noteworthy that the hyperbolic tangent activation function is adopted throughout this work, since our proposed methods enable the usage of smooth neural network models to reconstruct singular functions. As before, our trained model is expressed as $\widehat{G}(\boldsymbol{x}, \boldsymbol{y}, \varphi(\boldsymbol{x}, \boldsymbol{y}))$, while its projection (20) is denoted by $\check{G}(\boldsymbol{x}, \boldsymbol{y})$.



(a) V-cycle structures for different approaches



(b) solution errors over iterative cycles

Fig. 7. Multigrid methods with and without hybrid iterations for solving the discrete 1D Poisson Eq. (16).

Table 3
List of hyper-parameter configurations for learning the Green's function in our experiments.

Two-Dimensional Green's Function	Neural Network (Depth, Width)	Penalty Coeff. (β_{Snglr}, β_{Bndry}, β_{Symtr})	Train Data Size (N_R, N_S, N_B, M)	Total Training Epochs (Decay Milestones)
Poisson Eq. (16)	(4, 40)	(400, 400, 400)	(160, 500, 2, 500)	$20k$ ($12k$, $22k$)
Helmholtz Eq. (33)	(4, 40)	(400, 400, 400)	(500, 500, 2, 500)	$50k$ ($8k$, $15k$)
Conv-Diff Eq. (40)	(4, 40)	(400, 400, 400)	(400, 160, 2, 300)	$35k$ ($15k$, $20k$, $30k$)
Four-Dimensional Green's Function				
Poisson Eq. (36)	(6, 40)	(400, 400, 400)	(640, 200, 640, 160)	$30k$ ($15k$, $25k$)
Elliptic Eq. (43)	(6, 40)	(400, 400, 400)	(640, 200, 640, 160)	$30k$ ($15k$, $25k$)

All neural networks are trained via the ADAMW optimizer [64], using a step decay schedule that reduces the initial learning rate 10^{-3} by a factor of 10 after specific milestones for all experiments. The key hyperparameter configurations adopted in our practical implementations are summarized in Table 3. All experiments are conducted through PyTorch on the Nvidia GeForce RTX 4090 GPU cards. To ensure reproducibility, source codes are publicly accessible at https://github.com/q1sun/SgEncd_Green_Function.

5.1. Green's function for the helmholtz equation with a variable coefficient

Although initially developed for positive-definite problems (1), our singularity-encoded learning method remains effective when generalized to indefinite problems, as demonstrated through the Helmholtz equations

$$-(c(x)u'(x))' - k^2(x)u(x) = f(x), \qquad \text{for } x \in \Omega = (0,1), \tag{33}$$

with Dirichlet boundary conditions $u(0) = u(1) = 0$, variable coefficients $c(x) = (x-2)^2$ and $k(x) = 15\sin(10x)$. Accordingly, the exact Green's function $G(x,y)$ satisfies, for any $y \in \Omega$,

$$\begin{aligned} -c(x)\partial_{xx}G(x,y) - c'(x)\partial_x G(x,y) - k^2(x)G(x,y) &= 0, && \text{for } x \in \Omega \setminus \Gamma, \\ [\![G(x,y)]\!] = 0, \quad c(x)[\![\partial_x G(x,y)]\!] &= -1, && \text{for } x \in \Gamma = \{x \,|\, x = y\}, \\ G(x,y) &= 0, && \text{for } x \in \partial\Omega. \end{aligned} \tag{34}$$

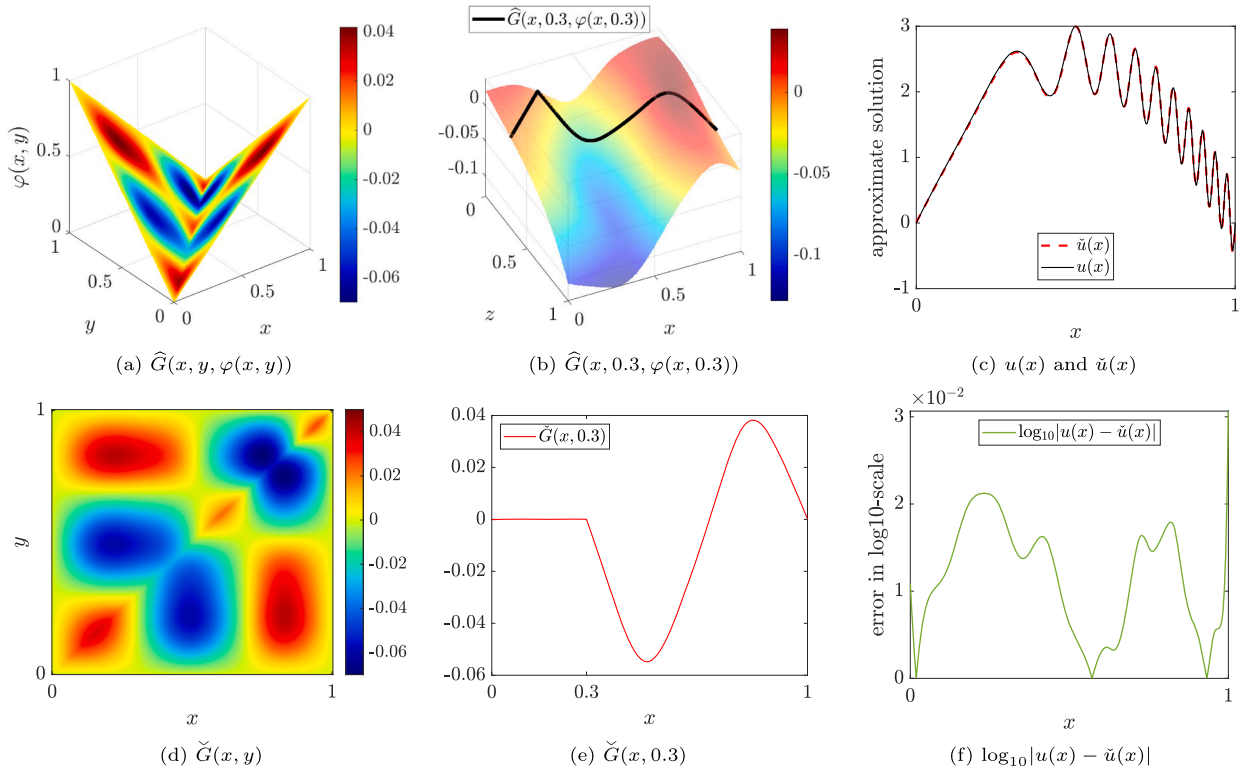
Unlike our illustrative example (17), the jump of derivative in (34) now incorporates a variable coefficient, which would complicate the derivation of the analytic Green's function [7,39]. Nevertheless, the exact Green's function maintains continuity across the interface Γ while exhibits jump in its derivative, hence our prescribed augmented variable $\varphi(x,y) = |x-y|$ remains valid in capturing the singular behavior governed by Eq. (34).

Then, by (12) and (11), the singularity-encoded Green's function $\widehat{G}(x,y,\varphi(x,y))$ for problem (34) satisfies

$$\begin{aligned} \big[c(x)\big(\partial_{xx} + 2\mathrm{sgn}(x-y)\partial_{xz} + \partial_{zz}\big) + c'(x)\big(\partial_x + \mathrm{sgn}(x-y)\partial_z\big) + k^2(x)\big]\widehat{G}(x,y,\varphi(x,y)) &= 0, && \text{for } x \in \Omega \setminus \Gamma, \\ -2c(x)\partial_z \widehat{G}(x,y,\varphi(x,y)) &= 1, && \text{for } x \in \Gamma, \\ \widehat{G}(x,y,\varphi(x,y)) &= 0, && \text{for } x \in \partial\Omega. \end{aligned} \tag{35}$$

for any fixed $y \in (0,1)$, while the constraint $0 = [\![G(x,y)]\!] = [\![\widehat{G}(x,y,\varphi(x,y))]\!]$ for $x \in \Gamma$ is automatically enforced owing to the continuity of our augmented variable $\varphi(x,y) = |x-y|$ along the interface $\Gamma = \{x \,|\, x = y\}$.

Next, the singularity-encoded Green's function is parametrized through a fully-connected neural network and learned via our proposed learning Algorithm 1. The trained model $\widehat{G}(x,y,\varphi(x,y))$ and its projection back onto the original 2-dimensional plane are shown in Fig. 8a and d, respectively. Moreover, by choosing $y = 0.3$, our approximate Green's function $\check{G}(x, y = 0.3)$ manifests a discontinuity in its first-order derivative as displayed in Fig. 8e, which can be effectively resolved due to its embedding into a smooth hyperplane $\widehat{G}(x, 0.3, z)$ within a one-order higher-dimensional space (see Fig. 8b).



(a) $\widehat{G}(x, y, \varphi(x, y))$ (b) $\widehat{G}(x, 0.3, \varphi(x, 0.3))$ (c) $u(x)$ and $\breve{u}(x)$

(d) $\breve{G}(x, y)$ (e) $\breve{G}(x, 0.3)$ (f) $\log_{10}|u(x) - \breve{u}(x)|$

Fig. 8. Numerical singularity-encoded Green's function for the 1D Helmholtz Eq. (33).

As the close-form Green's function is generally unavailable for problems with variable coefficients, we adopt the numerical integration (21) to evaluate the precision of our singularity-encoded Green's function. Specifically, with the forcing term $f(x)$ derived from the exact solution

$$u(x) = 10x - 10x^2 + 0.5\sin(20\pi x^3)$$

of Helmholtz Eq. (33), we use $h = 2^{-10}$ in (21) and compare our reconstructed solution $\breve{u}(x)$ with the exact solution $u(x)$. Fig. 8c and f demonstrate good agreement between numerical and true solutions, which implicitly implies that our approximate Green's function achieves satisfactory accuracy.

5.1.1. *Neural preconditioning matrix*

The Helmholtz Eq. (33) are discretized using a central difference scheme [8], leading to the linear system $AU = F$ (not relabelled for simplicity). Unlike the previous example (29), the discrete system becomes indefinite that undermines the effectiveness of traditional iterative methods (e.g., the damped Jacobi method). To guarantee convergence for indefinite systems, the generalized minimal residual method (abbreviated as GMRES) [43,65] is typically utilized with certain restart strategies. Nevertheless, ensuring efficient convergence remains challenging [65], necessitating the development and deployment of reliable preconditioners.

Building on the singularity-encoded Green's function offline-trained in our previous section, a preconditioner $\breve{B}$ can be constructed through Algorithm 2 for solving our indefinite systems. To be specific, the spectral property of our preconditioned matrix $\breve{B}A$ is computationally reported in Table 4, including the distribution of eigenvalues and the condition number across different mesh sizes. Our neural preconditioner, built from a surrogate model of its exact Green's function, produces clustered eigenvalues for the preconditioned system, which is crucial for the convergence of iterative methods. Subsequently, we evaluate the performance of GMRES (without restarting) for both the original (displayed in parentheses) and preconditioned systems, with comparative results reported in the final two columns of Table 4. Clearly, our preconditioned system can achieve satisfactory accuracy within only a few iterations, while the original system exhibits persistent error even after many iterations.

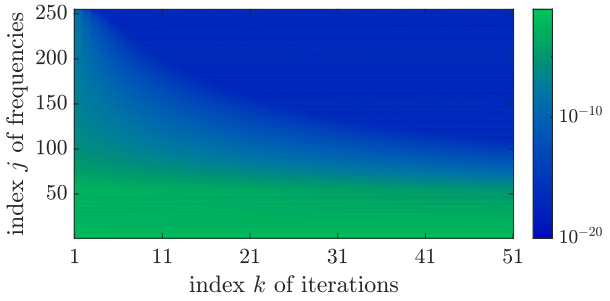
5.1.2. *Hybrid iterative methods*

Note that the indefiniteness of problem (33) causes many classical schemes, such as the Jacobi and Gauss-Seidel methods [43], to diverge when applied to solving its discrete linear system. Even for convergent components, the slow reduction of low-frequency errors undermines the overall efficiency. Here, we consider a linear system with mesh size $h = 2^{-8}$ for illustration, adopting the damped Jacobi method (31) with a relaxation parameter $\omega = 0.5$, and depict in Fig. 9a the mode-wise error across all frequencies.

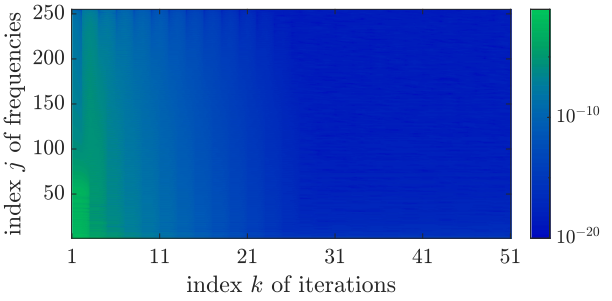
As can be seen from Fig. 9c, the mode-wise error $M^{[k],1}$ associated with the damped Jacobi method fails to converge due to the indefiniteness. Nevertheless, as shown in Fig. 9e, the rapid convergence of high-frequency errors remains unaffected. In contrast,

Table 4
GMRES methods with and without neural preconditioning for solving the discrete 1D Helmholtz Eq. (33).

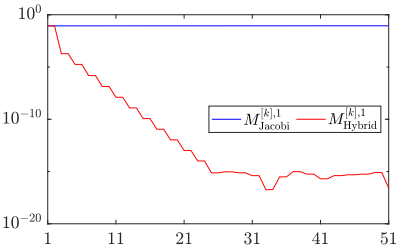
mesh size	eigenvalues of preconditioned matrix	condition number	# iterations	ℓ_2-norm error
$h = 2^{-6}$		$\kappa(\breve{B}A) = 1.03$	4 (30)	8.15×10^{-10} (4.92×10^{-1})
$h = 2^{-8}$		$\kappa(\breve{B}A) = 1.15$	5 (252)	3.85×10^{-10} (6.52×10^{-3})
$h = 2^{-10}$		$\kappa(\breve{B}A) = 1.69$	5 (1014)	6.82×10^{-9} (2.24×10^{-4})
$h = 2^{-12}$		$\kappa(\breve{B}A) = 3.85$	6 (4058)	1.55×10^{-10} (1.98×10^{-4})



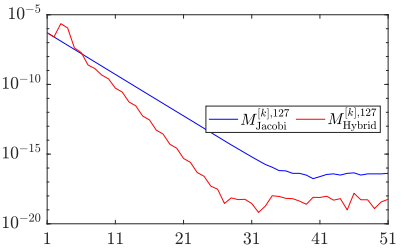
(a) the spectral error of damped Jacobi method (31)



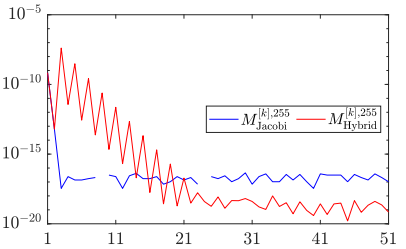
(b) the spectral error of hybrid iterative method (32)



(c) the lowest-frequency error component

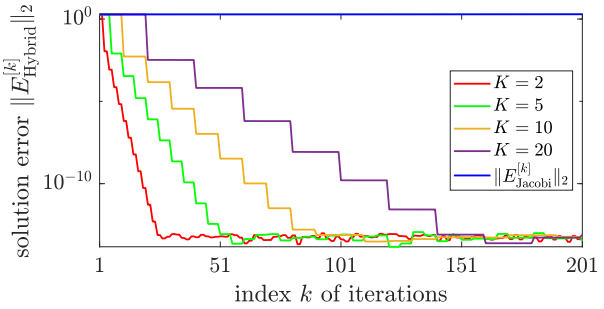


(d) the mid-frequency error component

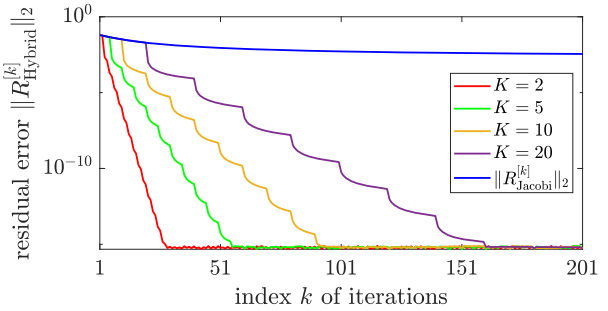


(e) the highest-frequency error component

Fig. 9. Comparison of spectral errors in iterative solvers for the discrete 1D Helmholtz Eq. (33).



(a) comparison of the solution error over iterations

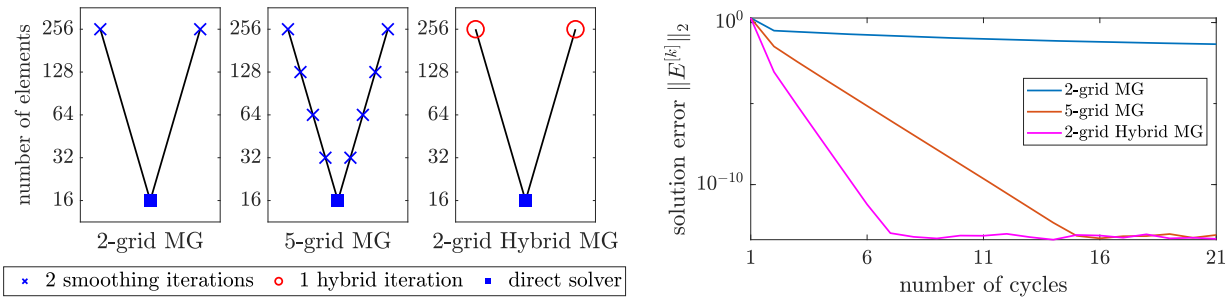


(b) comparison of the residual error over iterations

Fig. 10. Hybrid iteration (32) with different switching periods for the discrete 1D Helmholtz Eq. (33).

our hybrid iterative method (32) successfully resolves these limitations, as demonstrated in Fig. 9b and c. While the spectral bias inherent in our neural preconditioner introduces extra high-frequency errors (see Fig. 9e), these artifacts can be efficiently eliminated via traditional smoothing iterations. Moreover, empirical results in Fig. 10a and b imply that the switching period $K = 2$ achieves the minimum number of iterations required to attain convergence, whereas larger values impede the efficiency.

Then, we incorporate the hybrid iterative approach (32) within the multigrid framework, employing the same strategies as applied to the Poisson's equations in Section 4.2.1 (see also Fig. 11a for further details). Notably, applying our hybrid iterative scheme to both



(a) V-cycle structures for different approaches

(b) solution errors over iterative cycles

Fig. 11. Multigrid methods with and without hybrid iterations for solving the discrete 1D Helmholtz Eq. (33).

pre- and post-smoothing stages enables convergence to satisfactory accuracy in 7 cycles, even with the use of a two-grid structure as depicted in Fig. 11b.

5.2. Green's function for Poisson's equation in two dimension

We now extend our empirical studies to boundary value problems (1) in higher dimensions, in which the Green's function exhibits fundamentally different singular behavior (8) compared to the one-dimensional cases. Here, we consider the benchmark Poisson's equation on a two-dimensional unit disc, namely,

$$-\Delta u(\boldsymbol{x}) = f(\boldsymbol{x}), \quad \text{for } \boldsymbol{x} \in \Omega = \{\boldsymbol{x} : \|\boldsymbol{x}\| < 1\}, \tag{36}$$

where $\boldsymbol{x} = (x_1, x_2) \in \mathbb{R}^2$ and $u(\boldsymbol{x}) = 0$ for $\boldsymbol{x} \in \partial\Omega$. Accordingly, the Green's function for problem (36) satisfies

$$\begin{aligned} -\Delta G(\boldsymbol{x}, \boldsymbol{y}) &= \delta(\boldsymbol{x} - \boldsymbol{y}), && \text{for } \boldsymbol{x} \in \Omega, \\ G(\boldsymbol{x}, \boldsymbol{y}) &= 0, && \text{for } \boldsymbol{x} \in \partial\Omega, \end{aligned} \tag{37}$$

for any $\boldsymbol{y} = (y_1, y_2) \in \Omega$. By defining the point $\boldsymbol{y}^* = \frac{\boldsymbol{y}}{\|\boldsymbol{y}\|^2}$ dual to $\boldsymbol{y}$, the Green's function can be expressed as [66]

$$G(\boldsymbol{x}, \boldsymbol{y}) = -\frac{1}{2\pi} \ln\left(\frac{\|\boldsymbol{x} - \boldsymbol{y}\|}{\|\boldsymbol{x} - \boldsymbol{y}^*\|} \cdot \frac{1}{\|\boldsymbol{y}\|}\right) = -\frac{1}{4\pi} \ln\left(\frac{(x_1 - y_1)^2 + (x_2 - y_2)^2}{(x_1 y_2 - x_2 y_1)^2 + (x_1 y_1 + x_2 y_2 - 1)^2}\right), \tag{38}$$

which is highly expensive for traditional mesh-based methods to approximate due to the curse of dimensionality. Furthermore, the Green's function (38) exhibits a logarithmic blow-up, i.e., $G(\boldsymbol{x}, \boldsymbol{y}) \sim \ln\|\boldsymbol{x} - \boldsymbol{y}\|$ as $\boldsymbol{x} \to \boldsymbol{y}$, which introduces additional challenges for classical mesh-based numerical methods.

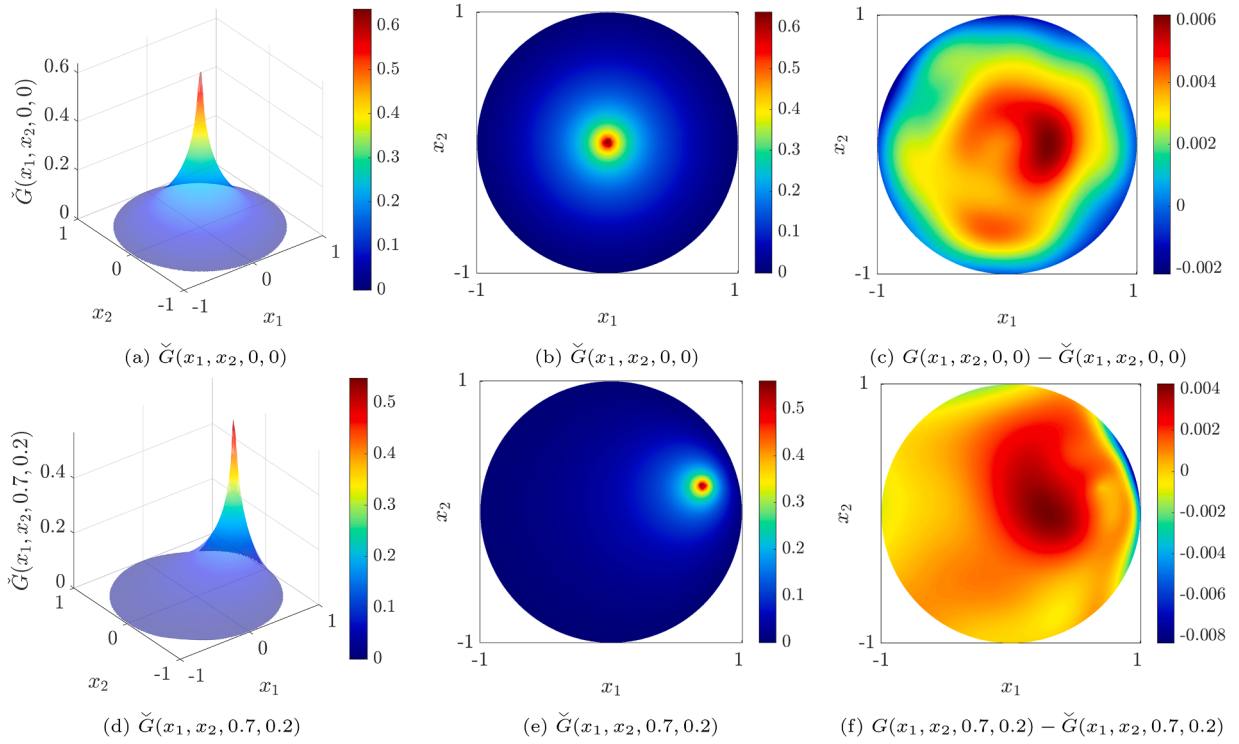
To address the logarithmic singularity, we augment the input variables of our deep surrogate model $\widehat{G}(\boldsymbol{x}, \boldsymbol{y}, \varphi(\boldsymbol{x}, \boldsymbol{y}))$ with an extra feature $\varphi(\boldsymbol{x}, \boldsymbol{y}) = \ln\|\boldsymbol{x} - \boldsymbol{y}\|$ as discussed in Section 3.1. Subsequently, by integrating the δ-function over some ball $B_\epsilon(\boldsymbol{y})$, formula (12) implies that our singularity-encoded Green's function satisfies, for any $\boldsymbol{y} \in \Omega$,

$$\begin{aligned} -\Delta_{\boldsymbol{x}} \widehat{G}(\boldsymbol{x}, \boldsymbol{y}, \varphi(\boldsymbol{x}, \boldsymbol{y})) - 2\nabla_{\boldsymbol{x}}\big(\partial_z \widehat{G}(\boldsymbol{x}, \boldsymbol{y}, \varphi(\boldsymbol{x}, \boldsymbol{y}))\big) \cdot \frac{\boldsymbol{x} - \boldsymbol{y}}{\|\boldsymbol{x} - \boldsymbol{y}\|^2} - \frac{\partial_{zz} \widehat{G}(\boldsymbol{x}, \boldsymbol{y}, \varphi(\boldsymbol{x}, \boldsymbol{y}))}{\|\boldsymbol{x} - \boldsymbol{y}\|^2} = 0, \quad & \text{for } \boldsymbol{x} \in \Omega \setminus \Gamma, \\ -\int_{\partial B_\epsilon(\boldsymbol{y})} \left(\nabla_{\boldsymbol{x}} G(\boldsymbol{x}, \boldsymbol{y}, \varphi(\boldsymbol{x}, \boldsymbol{y})) \cdot \frac{\boldsymbol{x} - \boldsymbol{y}}{\|\boldsymbol{x} - \boldsymbol{y}\|} + \frac{\partial_z \widehat{G}(\boldsymbol{x}, \boldsymbol{y}, \varphi(\boldsymbol{x}, \boldsymbol{y}))}{\|\boldsymbol{x} - \boldsymbol{y}\|}\right) dS(\boldsymbol{x}) = 1, \quad & \text{for } \boldsymbol{x} \in \Gamma, \\ G(\boldsymbol{x}, \boldsymbol{y}, \varphi(\boldsymbol{x}, \boldsymbol{y})) = 0, \quad & \text{for } \boldsymbol{x} \in \partial\Omega, \end{aligned} \tag{39}$$

where $\Gamma = \{\boldsymbol{x} \mid \boldsymbol{x} = \boldsymbol{y}\}$. Notably, as discussed in (13), our singularity-encoded Green's function (39) now exhibits bounded partial derivatives $\nabla_{\boldsymbol{x}} \widehat{G}(\boldsymbol{x}, \boldsymbol{y}, \varphi(\boldsymbol{x}, \boldsymbol{y}))$ and $\partial_z \widehat{G}(\boldsymbol{x}, \boldsymbol{y}, \varphi(\boldsymbol{x}, \boldsymbol{y}))$, which differs from existing approaches (see Section 2.1) but requires computation within a five-dimensional space. Fortunately, artificial neural networks [2] have demonstrated remarkable effectiveness in handling high-dimensional smooth functions.

Consequently, we employ a fully-connected neural network with Tanh activation functions to parametrize the singularity-encoded Green's function, which is then optimized via Algorithm 1. To capture the singular behavior governed by our normalization condition (39), we adopt four concentric circles $B_\epsilon(\boldsymbol{y}_m)$ for each sampled $\boldsymbol{y}_m \in \Gamma$, with the radius chosen to be $\epsilon = 0.1$, 0.08, 0.064, and 0.005. To visualize our numerical results, two slices of the trained neural network are depicted in Fig. 12a and d by fixing source points at $\boldsymbol{y} = (0, 0)$ and $(0.7, 0.2)$ respectively. In addition, Fig. 12c and f depict the discrepancies between the true Green's function (38) and our surrogate model for these two cases, demonstrating satisfactory resolution of the logarithmic singularity near $\boldsymbol{x} = \boldsymbol{y}$. As a direct application, our singularity-encoded Green's function is used to reconstruct the solution

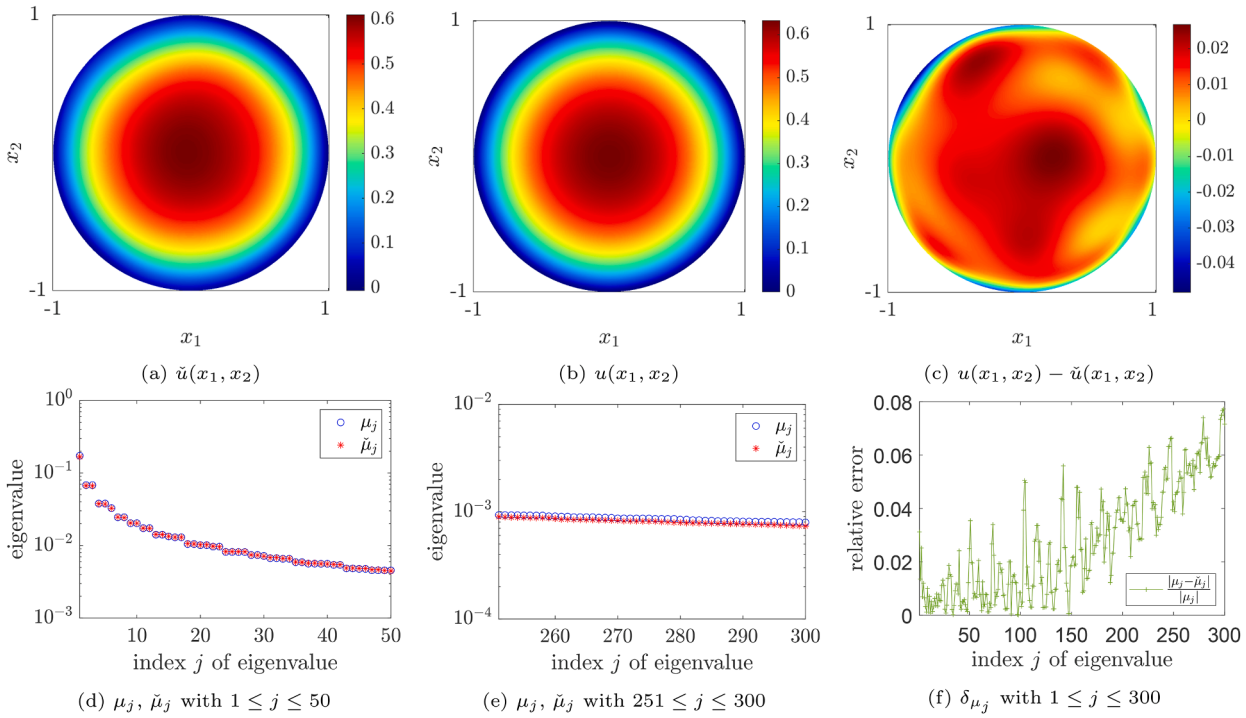
$$u(\boldsymbol{x}) = -e^{x_1^2 + x_2^2 - 1} + 1$$



(a) $\breve{G}(x_1, x_2, 0, 0)$ (b) $\breve{G}(x_1, x_2, 0, 0)$ (c) $G(x_1, x_2, 0, 0) - \breve{G}(x_1, x_2, 0, 0)$

(d) $\breve{G}(x_1, x_2, 0.7, 0.2)$ (e) $\breve{G}(x_1, x_2, 0.7, 0.2)$ (f) $G(x_1, x_2, 0.7, 0.2) - \breve{G}(x_1, x_2, 0.7, 0.2)$

Fig. 12. Numerical singularity-encoded Green's function for the 2D Poisson Eq. (36).

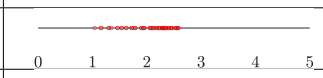
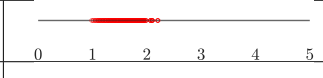
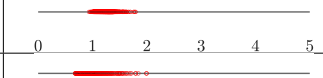
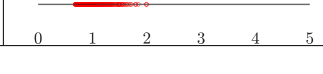


(a) $\breve{u}(x_1, x_2)$ (b) $u(x_1, x_2)$ (c) $u(x_1, x_2) - \breve{u}(x_1, x_2)$

(d) μ_j, $\breve{\mu}_j$ with $1 \le j \le 50$ (e) μ_j, $\breve{\mu}_j$ with $251 \le j \le 300$ (f) δ_{μ_j} with $1 \le j \le 300$

Fig. 13. Numerical validation via fast solver and spectral comparison for the 2D Poisson Eq. (36).

Table 5
BiCG methods with and without neural preconditioning for solving the discrete 2D Poisson Eq. (36).

mesh size	eigenvalues of preconditioned matrix	condition number	# iterations	ℓ_2-norm error
$h = 0.1$		$\kappa(\breve{B}A) = 2.48$ $\kappa(A) = 20.55$	17 (27)	2.41×10^{-14} (4.04×10^{-14})
$h = 0.05$		$\kappa(\breve{B}A) = 2.25$ $\kappa(A) = 89.68$	17 (59)	3.01×10^{-14} (3.34×10^{-14})
$h = 0.025$		$\kappa(\breve{B}A) = 1.92$ $\kappa(A) = 446.16$	16 (129)	2.51×10^{-15} (6.06×10^{-14})
$h = 0.0125$		$\kappa(\breve{B}A) = 2.96$ $\kappa(A) = 2041.57$	22 (282)	9.46×10^{-15} (1.08×10^{-14})

through the numerical integration (21), bypassing the need to numerically solve our Poisson's Eq. (36). The pointwise error depicted in Fig. 13c implies close agreement between the exact and reconstructed solutions.

Particularly, eigenvalues of the exact Green's function (38) are available for this specific problem [67], hence enabling a direct and rigorous comparison of those computed from our singularity-encoded Green's function. By using linear elements with mesh width $h = 0.045$ in formula (23), we depict in Fig. 13d and e the exact eigenvalues and our approximated counterparts across a wide range of frequencies, along with their relative error in Fig. 13f. Our numerical results reveal a clear manifestation of spectral bias [48], where low-frequency modes are reconstructed with high accuracy but high-frequency counterparts progressively deteriorates.

5.2.1. Neural preconditioning matrix

Based on our pre-trained singularity-encoded Green's function, a neural preconditioner for the discrete system of problem (36) can be constructed by applying Algorithm 2. The discretization is conducted using linear elements, while spectral properties of our preconditioned system are computationally evaluated and reported in Table 5. As in the one-dimensional case (see Section 4.1.1), the discrete linear system is iteratively solved via the biconjugate gradient method [43], both with and without the use of our neural preconditioner.

As expected, the condition number of the original stiffness matrix grows with mesh refinement, leading to an increasing number of iterations to achieve convergence at machine precision. On the contrary, eigenvalues of our preconditioned matrix are notably clustered, allowing comparable accuracy to be achieved with fewer iterations.

5.2.2. Hybrid iterative methods

To exploit the spectral bias inherent in our singularity-encoded Green's function, the hybrid iterative scheme (32) is applied to solve the discrete system of (36) with mesh size $h = 0.025$, followed by the comparison with that of the damped Jacobi method (31). As evident in Fig. 14a and b, the damped Jacobi method is effective in attenuating high-frequency errors but exhibits limited efficacy in eliminating low-frequency errors, as reported in Section 4.2.1. On the contrary, our hybrid method provides rapid error reduction across all frequencies, achieving machine accuracy within approximately 50 no. of iterations. Moreover, Fig. 15a and b suggest that $K = 2$ remains the most efficient choice of switching period, consistent with our findings in the one dimension.

Next, a comparative study between classical multigrid methods and our hybrid-smoothing variants are shown in Fig. 16a and b. Remarkably, the two-grid variant achieves a faster convergence than the traditional four-grid scheme, reducing the reliance on deeply nested grids without compromising solution accuracy.

5.3. Green's function for the convection-diffusion equation in one dimension

In addition to the symmetric operators previously studied, the proposed learning algorithm is also well-suited for nonsymmetric problems. To demonstrate this versatility, we consider the one-dimensional convection-diffusion equation

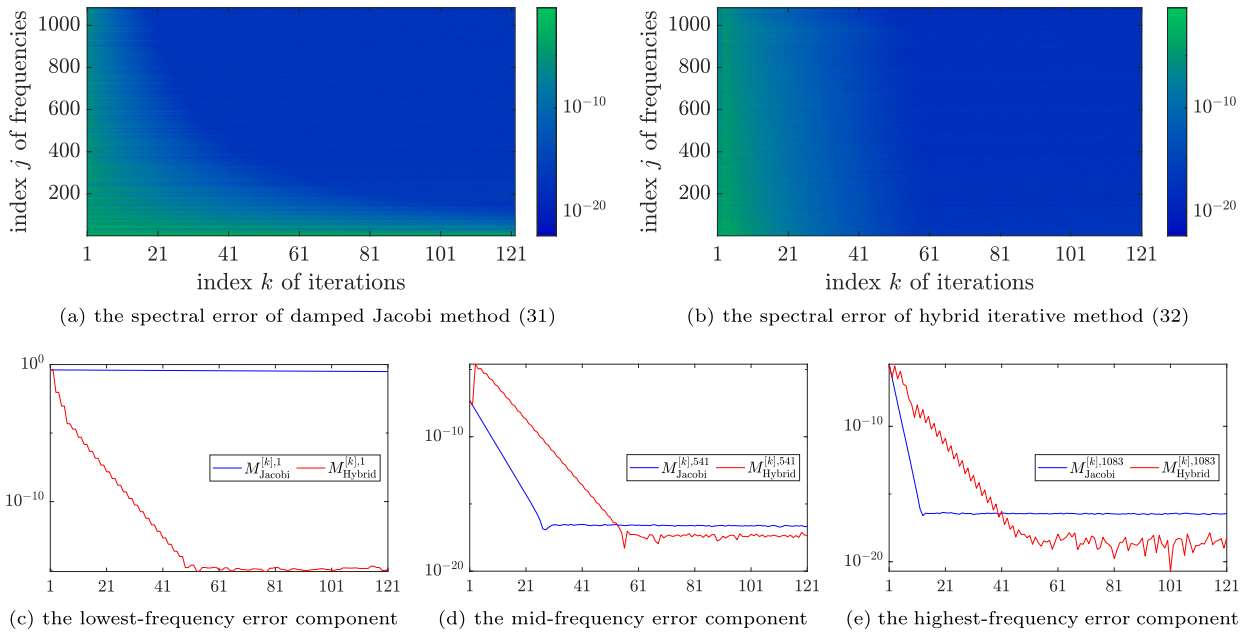
$$\frac{1}{4}u''(x) + \frac{1}{2}u'(x) + \frac{1}{4}u(x) = f(x), \qquad \text{for } x \in \Omega = (0,1), \tag{40}$$

subject to the Dirichlet boundary conditions $u(0) = u(1) = 0$. Accordingly, for any impulse source located at $y \in \Omega$, the boundary value problem (3) defining its Green's function can be equivalently rewritten as

$$\begin{aligned} \frac{1}{4}\partial_{xx}G(x,y) + \frac{1}{2}\partial_x G(x,y) + \frac{1}{4}G(x,y) = 0, &\qquad \text{for } x \in \Omega \setminus \Gamma, \\ [\![G(x,y)]\!] = 0, \quad [\![\partial_x G(x,y)]\!] = 4, &\qquad \text{for } x \in \Gamma = \{x \mid x = y\}, \\ G(x,y) = 0, &\qquad \text{for } x \in \partial\Omega. \end{aligned} \tag{41}$$

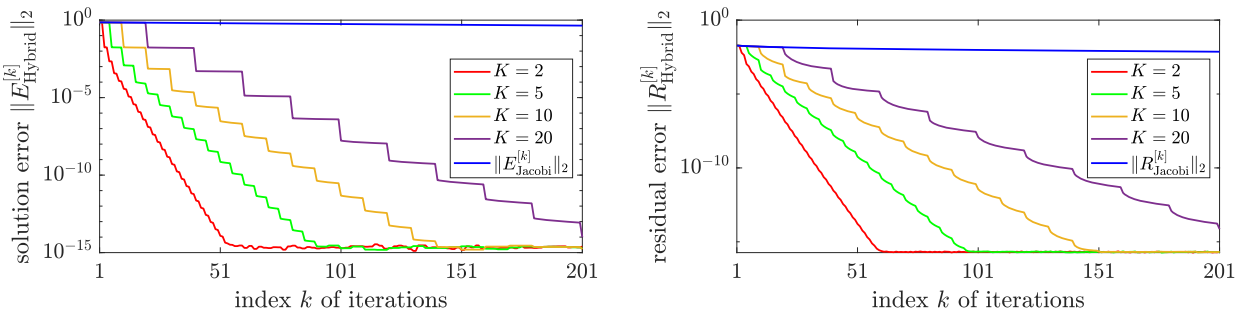
with the analytic solution given by

$$G(x,y) = \begin{cases} 4x(y-1)e^{-(x-y)}, & \text{for } 0 \le x \le y, \\ 4y(x-1)e^{-(x-y)}, & \text{for } y \le x \le 1. \end{cases}$$



(a) the spectral error of damped Jacobi method (31)
(b) the spectral error of hybrid iterative method (32)
(c) the lowest-frequency error component
(d) the mid-frequency error component
(e) the highest-frequency error component

Fig. 14. Comparison of spectral errors in iterative solvers for the discrete 2D Poisson Eq. (36).



(a) comparison of the solution error over iterations
(b) comparison of the residual error over iterations

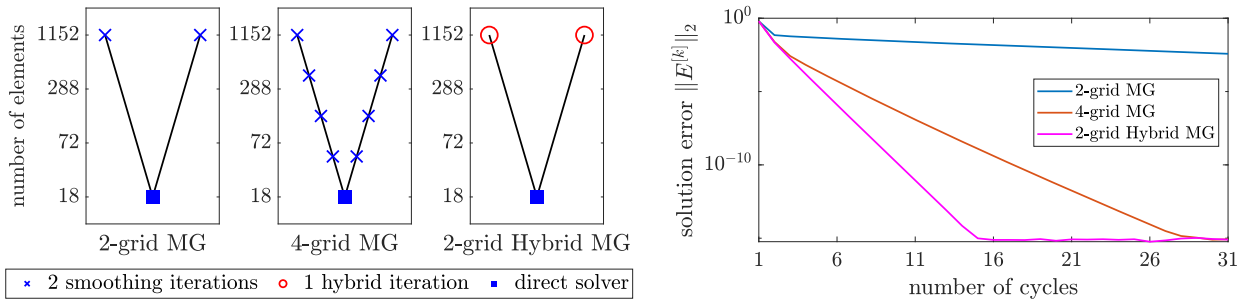
Fig. 15. Hybrid iteration (32) with different switching periods for the discrete 2D Poisson Eq. (36).

Despite the lack of symmetry in the elliptic operator (40), the singularity of the Green's function (41) remains governed by the jump in its first order derivative across the interface Γ, ensuing the effectiveness of our augmented variable $\varphi(x, y) = |x - y|$ for singularity encoding. Then, by (11) and (12), the singularity-encoded Green's function for problem (40) satisfies, for any fixed $y \in \Omega$,

$$\begin{aligned}
\left[\frac{1}{4}\left(\partial_{xx} + 2\text{sgn}(x-y)\partial_{xz} + \partial_{zz}\right) + \frac{1}{2}\left(\partial_x + \text{sgn}(x-y)\partial_z\right) + \frac{1}{4}\right]\widehat{G}(x, y, \varphi(x, y)) = 0, \quad & \text{for } x \in \Omega \setminus \Gamma, \\
\partial_z \widehat{G}(x, y, \varphi(x, y)) = 2, \quad & \text{for } x \in \Gamma, \\
\widehat{G}(x, y, \varphi(x, y)) = 0, \quad & \text{for } x \in \partial\Omega.
\end{aligned} \tag{42}$$

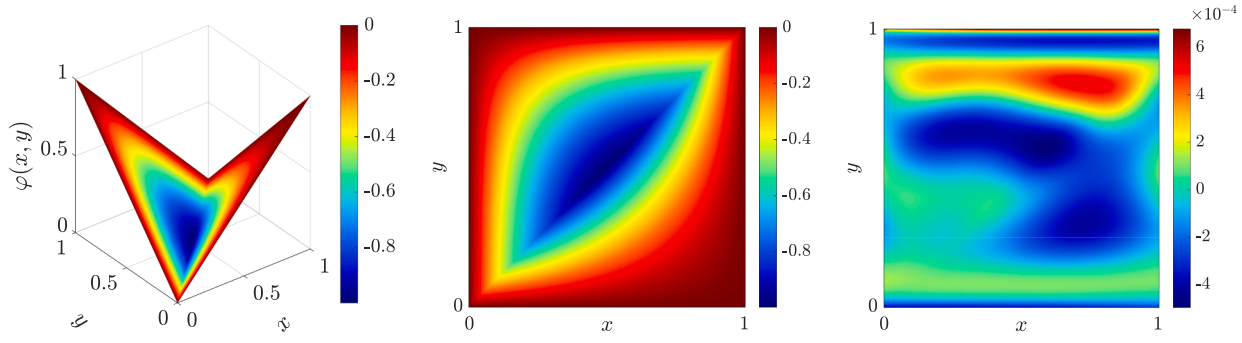
As in the previous cases, a fully connected neural network with smooth activation functions is used to parametrize the solution ansatz $\widehat{G}(x, y, \varphi(x, y))$ for problem (42), followed by training through Algorithm 1 with the indicator function set to zero to disable the symmetry penalty. The trained model solution and its projection onto the physical domain are depicted in Fig. 17a and b respectively, while the pointwise error compared to the exact solution in Fig. 17c demonstrates satisfactory agreement.

In addition, our surrogate Green's function is employed as a fast solver (21) to reconstruct the solution $u(x) = 10x - 10x^2 + 0.5\sin(20\pi x^3)$ for a source term derived from (40), with numerical results illustrated in Fig. 18b and c exhibiting satisfactory numerical consistency.



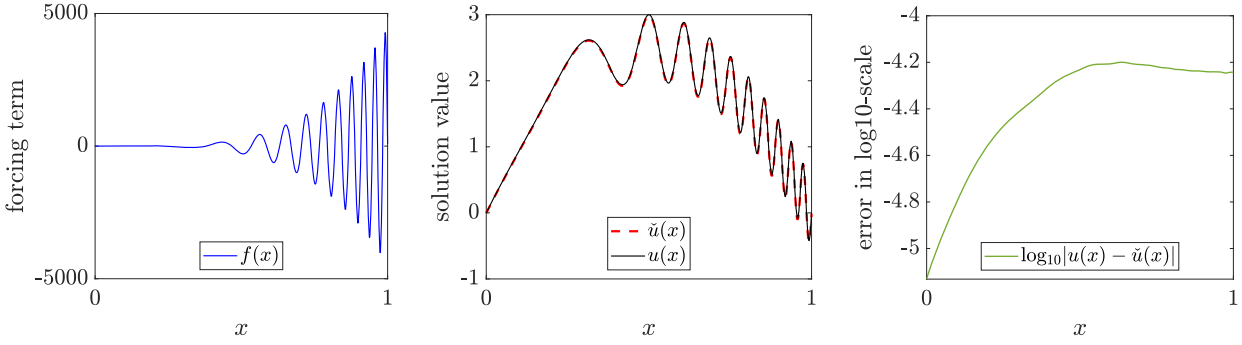
(a) V-cycle structures for different approaches (b) solution errors over iterative cycles

Fig. 16. Multigrid methods with and without hybrid iterations for solving the discrete 2D Poisson Eq. (36).



(a) $\widehat{G}(x, y, \varphi(x, y))$ (b) $\breve{G}(x, y)$ (c) $G(x, y) - \breve{G}(x, y)$

Fig. 17. Numerical singularity-encoded Green's function for the 1D convection-diffusion Eq. (40).



(a) $f(x)$ (b) $u(x)$ and $\breve{u}(x)$ (c) $\log_{10}|u(x) - \breve{u}(x)|$

Fig. 18. Numerical validation via fast solver for the 1D convection-diffusion Eq. (40).

5.4. *Green's function for variable-coefficient elliptic equations on a flower-shaped domain*

Extending beyond simple geometries, the meshfree nature of neural networks facilitates the treatment of complex domains. To demonstrate this capability, we consider a variable-coefficient elliptic equation

$$\begin{aligned} -\nabla \cdot ((1 + x_1^2)\nabla u(\boldsymbol{x})) &= f(\boldsymbol{x}), && \text{for } \boldsymbol{x} \in \Omega, \\ u(\boldsymbol{x}) &= 0, && \text{for } \boldsymbol{x} \in \partial\Omega. \end{aligned} \tag{43}$$

posed on a flower-shaped domain in two dimension

$$\Omega = \left\{ \boldsymbol{x} = (r\cos\theta,\, r\sin\theta) \in \mathbb{R}^2 \mid 0 \le r < 0.8 + 0.15\cos(5\theta),\ \theta \in [0, 2\pi) \right\}.$$

Owing to the coercivity of the elliptic operator (43), the associated Green's function preserves a logarithmic singularity, thereby ensuring the use of augmented variable $\varphi(\boldsymbol{x}, \boldsymbol{y}) = \ln \|\boldsymbol{x} - \boldsymbol{y}\|$ in the construction of our solution ansatz $\widehat{G}(\boldsymbol{x}, \boldsymbol{y}, \varphi(\boldsymbol{x}, \boldsymbol{y}))$. By defining

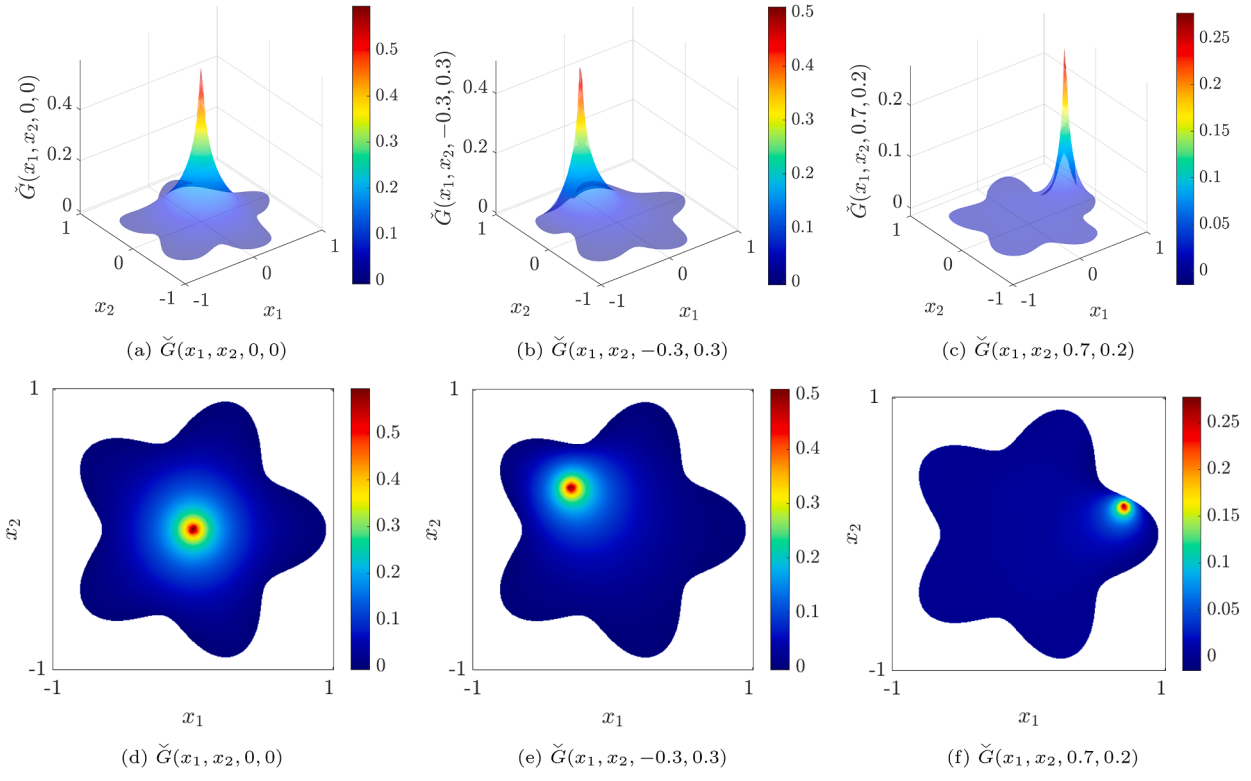


Fig. 19. Numerical singularity-encoded Green's function for the 2D elliptic Eq. (43).

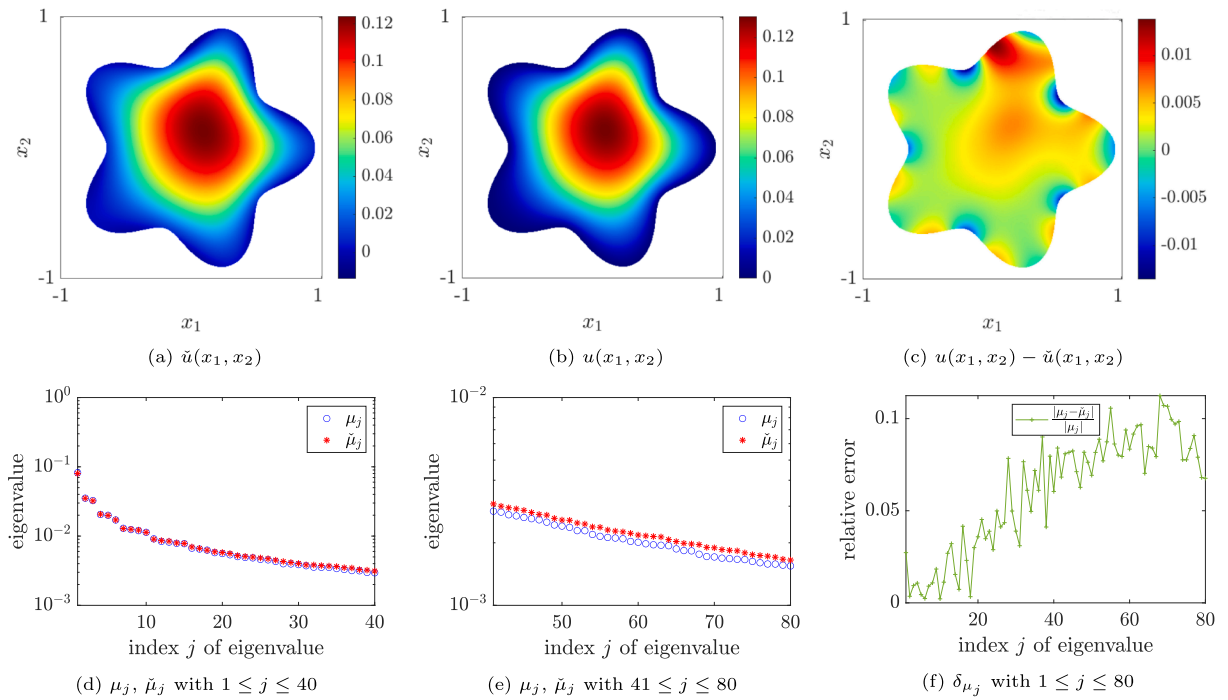


Fig. 20. Numerical validation via fast solver and spectral comparison for the 2D elliptic Eq. (43).

$\Gamma = \{\boldsymbol{x} = \boldsymbol{y}\}$, a direct calculation implies that the system (12) governing our singularity-encoded Green's function takes on the form

$$\begin{aligned} -2x_1 \cdot \left(\nabla_{\boldsymbol{x}}\widehat{G} + \nabla\varphi\partial_z\widehat{G}\right) - (1+x_1^2)\left(\Delta_{\boldsymbol{x}}\widehat{G} + 2\nabla\varphi\cdot\nabla_{\boldsymbol{x}}(\partial_z\widehat{G}) + \|\nabla\varphi\|^2\partial_{zz}\widehat{G}\right) = 0, \quad & \text{for } \boldsymbol{x}\in\Omega\setminus\Gamma, \\ -\int_{\partial B_\epsilon(\boldsymbol{y})} (1+x_1^2)\left(\nabla_{\boldsymbol{x}}\widehat{G} + \nabla\varphi\partial_z\widehat{G}\right)\cdot\boldsymbol{n}_{\boldsymbol{x}}\, dS(\boldsymbol{x}) = 1, \quad & \text{for } \boldsymbol{x}\in\Gamma, \\ \widehat{G} = 0, \quad & \text{for } \boldsymbol{x}\in\partial\Omega. \end{aligned} \tag{44}$$

for any fixed source point $\boldsymbol{y}\in\Omega$ and any $\epsilon > 0$. In contrast to the Poisson's Eq. (36) in Section 5.2, an analytical expression for the Green's function of problem (43) is unavailable due to the presence of variable coefficient and the complexity of the domain geometry.

For the numerical implementation via Algorithm 1, we adopt a standard fully connected neural network, with key hyperparameter configurations reported in Table 3. Notably, the normalization condition in (44) is enforced across a sequence of diminishing radius parameters, $\epsilon \in \{0.1, 0.08, 0.064, 0.005\}$ as in Section 5.2, to approximate the asymptotic singular behavior. By fixing source points at $(0,0)$, $(-0.3, 0.3)$, and $(0.7, 0.2)$, we visualize the corresponding solution slices in Fig. 19a, b, and c, respectively.

In the absence of a closed-form expression, our approximated Green's function is validated by recovering the solution of problem (43) for the source term $f(\boldsymbol{x}) = 1 + x_1 + x_2$ through the integral representation (21), where the reference solution shown in Fig. 20b is obtained by numerically solving (43) via linear elements with mesh size $h = 0.03$. The pointwise error in Fig. 13c demonstrates the reliability of the reconstructed solution, thereby validating the effectiveness of our trained neural network solution.

Moreover, the eigenvalues of problem (43) are computed numerically, with their reciprocals serving as reference spectra for the exact Green's function. This facilitates a rigorous evaluation of the numerical fidelity and enables a direct comparison with the spectrum computed from the Mercer's theorem, where Fig. 20f reveals a spectral bias in our trained ansatz that aligns with prior observations.

6. Conclusions

In this paper, we propose a singularity-encoded learning framework to reconstruct the Green's function of elliptic boundary value problems without supervised training data. Based on the estimate of its singularities, our Green's function is embedded into a one-order higher-dimensional space by incorporating an augmented variable, leading to a numerically tractable surrogate with improved regularity. This enables us to reconstruct the Green's function using fully-connected neural networks with smooth activation functions, while the sampling of collocation points remains unchanged regardless of the increased dimensionality. After projecting the trained neural network model back onto its physical domain, the spectral bias inherent to our singularity-encoded Green's function is applied to accelerate classical iterative solvers, either as an effective preconditioner or as a correction step to eliminate low- frequency errors. Numerical studies across benchmark problems are reported to demonstrate the effectiveness and efficiency of our proposed methods.

Our proposed methods offer a promising framework for integrating deep learning techniques with traditional numerical schemes and the rigorous theoretical analysis will be the focus of future research. It is also noteworthy that the proposed singularity-encoded technique admits a natural extension to a broad class of singular problems, a direction that will be explored in our subsequent work. Finally, the integration of equation-driven and data-driven methods for solving the Green's functions (and singular functions more broadly) also constitutes a highly valuable research direction.

CRediT authorship contribution statement

Qi Sun: Writing – review & editing, Writing – original draft, Supervision, Software, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization; **Shengyan Li:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis; **Bowen Zheng:** Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis; **Lili Ju:** Writing – review & editing, Supervision, Methodology, Investigation; **Xuejun Xu:** Writing – review & editing, Supervision, Methodology, Funding acquisition.

Data availability

No data was used for the research described in the article.

Declaration of competing interest

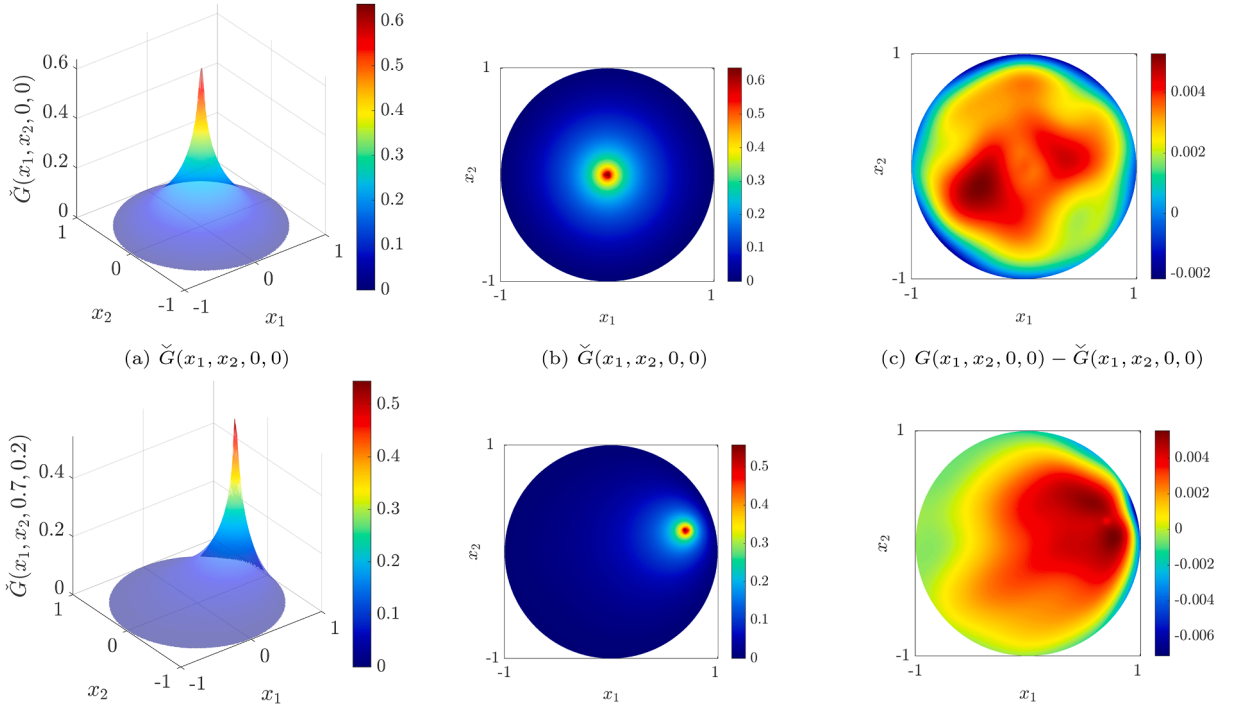
None.

Acknowledgements

The authors thank the anonymous reviewers for their constructive comments and suggestions and Professor Qiang Du of Columbia University for his valuable discussions. The computations were done on the high performance computers of School of Mathematical Sciences, Tongji University.

Appendix A. Green's function for poisson's equation in two dimension

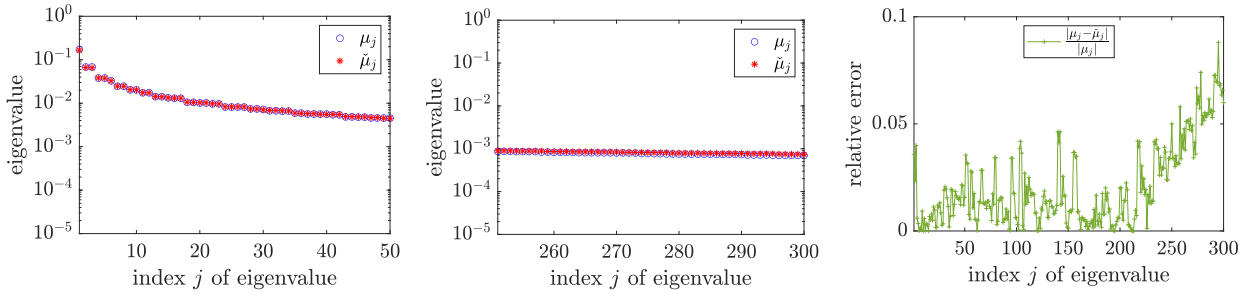
Unlike the logarithmic augmented variable (9), we now consider an alternative approach of the form $\varphi(\boldsymbol{x}, \boldsymbol{y}) = \|\boldsymbol{x} - \boldsymbol{y}\|^{-0.2}$ for (12). Numerical results for the four-dimensional problem (37) are displayed in Figs. A.1 and A.2, which imply that our singularity-encoded Green's function achieves satisfactory accuracy. Specifically, the realizations of our singularity-encoded Green's function at fixed points $\boldsymbol{y} = (0, 0)$ and $\boldsymbol{y} = (0.7, 0.2)$ are depicted in Fig. A.1a and d, along with their error profiles in Fig. A.1c and f. Moreover, the comparison between exact and numerical eigenvalues are presented in Fig. A.2a, b, and c, demonstrating the spectral bias of our trained model.



(a) $\breve{G}(x_1, x_2, 0, 0)$ (b) $\breve{G}(x_1, x_2, 0, 0)$ (c) $G(x_1, x_2, 0, 0) - \breve{G}(x_1, x_2, 0, 0)$

(d) $\breve{G}(x_1, x_2, 0.7, 0.2)$ (e) $\breve{G}(x_1, x_2, 0.7, 0.2)$ (f) $G(x_1, x_2, 0.7, 0.2) - \breve{G}(x_1, x_2, 0.7, 0.2)$

Fig. A.1. Numerical singularity-encoded Green's function for the 2D Poisson problem (36) with $\varphi(\boldsymbol{x}, \boldsymbol{y}) = \|\boldsymbol{x} - \boldsymbol{y}\|^{-0.2}$.



(a) μ_j, $\breve{\mu}_j$ with $1 \le j \le 50$ (b) μ_j, $\breve{\mu}_j$ with $251 \le j \le 300$ (c) δ_{μ_j} with $1 \le j \le 300$

Fig. A.2. Numerical validation via spectral comparison for the 2D Poisson problem (36) with $\varphi(\boldsymbol{x}, \boldsymbol{y}) = \|\boldsymbol{x} - \boldsymbol{y}\|^{-0.2}$.

Appendix B. Proof of Remark 2

Here we give a proof for the result presented in Remark 2.

Theorem 1. *For each fixed point $\boldsymbol{y} \in \Omega$, let us define $G(\cdot, \boldsymbol{y}) \in W^{1,1}(\Omega)$ (not relabelled) as the function satisfying*

$$\begin{aligned} -\nabla \cdot (c(\boldsymbol{x})\nabla G(\boldsymbol{x}, \boldsymbol{y})) &= 0, && \text{for } \boldsymbol{x} \in \Omega \setminus \{\boldsymbol{y}\}, \\ -\int_{\partial B_\epsilon(\boldsymbol{y})} c(\boldsymbol{x})\nabla G(\boldsymbol{x}, \boldsymbol{y}) \cdot \boldsymbol{n}_x \, dS(\boldsymbol{x}) &= 1, && \text{for any } \epsilon > 0, \\ G(\boldsymbol{x}, \boldsymbol{y}) &= 0, && \text{for } \boldsymbol{x} \in \partial\Omega. \end{aligned} \tag{B.1}$$

Then, in the sense of distributions, it follows that

$$\begin{aligned} -\nabla \cdot (c(\boldsymbol{x})\nabla G(\boldsymbol{x}, \boldsymbol{y})) &= \delta(\boldsymbol{x} - \boldsymbol{y}), && \text{for } \boldsymbol{x} \in \Omega, \\ G(\boldsymbol{x}, \boldsymbol{y}) &= 0, && \text{for } \boldsymbol{x} \in \partial\Omega. \end{aligned} \tag{B.2}$$

Proof. By the self-adjointness of the elliptic operator $\mathcal{L}u = -\nabla \cdot (c(\boldsymbol{x})\nabla u(\boldsymbol{x}))$, it is equivalent to show that for any test function $\phi \in C_c^\infty(\Omega)$, the following equality holds:

$$\int_\Omega G(\boldsymbol{x}, \boldsymbol{y})\mathcal{L}\phi(\boldsymbol{x}) \, d\boldsymbol{x} = \phi(\boldsymbol{y}). \tag{B.3}$$

Note that the function $G(\cdot, \boldsymbol{y})$ satisfying (B.1) is singular at the point $\boldsymbol{y} \in \Omega$, as a removable singularity would contradict the flux normalization condition. The integral in (B.3) is then formulated as the limit of an integral over the domain excluding a small ball $B_\epsilon(\boldsymbol{y})$, that is,

$$\int_\Omega G(\boldsymbol{x}, \boldsymbol{y})\mathcal{L}\phi(\boldsymbol{x}) \, d\boldsymbol{x} = \lim_{\epsilon \to 0} \int_{\Omega \setminus B_\epsilon(\boldsymbol{y})} G(\boldsymbol{x}, \boldsymbol{y})\mathcal{L}\phi(\boldsymbol{x}) \, d\boldsymbol{x}.$$

On the other hand, the Green's second identity on the punctured domain $\Omega_\epsilon = \Omega \setminus B_\epsilon(\boldsymbol{y})$ implies

$$\int_{\Omega_\epsilon} G(\boldsymbol{x}, \boldsymbol{y})\mathcal{L}\phi(\boldsymbol{x}) - \phi(\boldsymbol{x})\mathcal{L}G(\boldsymbol{x}, \boldsymbol{y}) \, d\boldsymbol{x} = \int_{\partial\Omega_\epsilon} c(\boldsymbol{x})\big[\phi(\boldsymbol{x})\nabla G(\boldsymbol{x}, \boldsymbol{y}) - G(\boldsymbol{x}, \boldsymbol{y})\nabla\phi(\boldsymbol{x})\big] \cdot \boldsymbol{n} \, dS(\boldsymbol{x})$$

which simplifies to

$$\int_{\Omega_\epsilon} G(\boldsymbol{x}, \boldsymbol{y})\mathcal{L}\phi(\boldsymbol{x}) \, d\boldsymbol{x} = \int_{\partial B_\epsilon(\boldsymbol{y})} c(\boldsymbol{x})\big[G(\boldsymbol{x}, \boldsymbol{y})\nabla\phi(\boldsymbol{x}) - \phi(\boldsymbol{x})\nabla G(\boldsymbol{x}, \boldsymbol{y})\big] \cdot \boldsymbol{n}_{\boldsymbol{x}} \, dS(\boldsymbol{x})$$

upon application of the equation $\mathcal{L}G(\cdot, \boldsymbol{y}) = 0$ in $\Omega \setminus \{\boldsymbol{y}\}$ and boundary conditions $\phi|_{\partial\Omega} = G(\cdot, \boldsymbol{y})|_{\partial\Omega} = 0$. Here, $\boldsymbol{n}_{\boldsymbol{x}} = \frac{\boldsymbol{x}-\boldsymbol{y}}{\|\boldsymbol{x}-\boldsymbol{y}\|} = -\boldsymbol{n}|_{\partial B_\epsilon(\boldsymbol{y})}$ is the unit outer normal vector along the boundary $\partial B_\epsilon(\mathbf{y})$, in accordance with the notation used in (12). Therefore, it remains to prove that

$$\lim_{\epsilon \to 0} \int_{\partial B_\epsilon(\boldsymbol{y})} c(\boldsymbol{x})\big[G(\boldsymbol{x}, \boldsymbol{y})\nabla\phi(\boldsymbol{x}) - \phi(\boldsymbol{x})\nabla G(\boldsymbol{x}, \boldsymbol{y})\big] \cdot \boldsymbol{n}_{\boldsymbol{x}} \, dS(\boldsymbol{x}) = \phi(\boldsymbol{y}). \tag{B.4}$$

Moreover, the normalization condition in (B.1) implies that

$$\phi(\boldsymbol{y}) = -\phi(\boldsymbol{y}) \int_{\partial B_\epsilon(\boldsymbol{y})} c(\boldsymbol{x})\nabla G(\boldsymbol{x}, \boldsymbol{y}) \cdot \boldsymbol{n}_{\boldsymbol{x}} \, dS(\boldsymbol{x})$$

for any $\epsilon > 0$, hence it is sufficient to prove

$$I_1 = \lim_{\epsilon \to 0} \int_{\partial B_\epsilon(\boldsymbol{y})} c(\boldsymbol{x})G(\boldsymbol{x}, \boldsymbol{y})\nabla\phi(\boldsymbol{x}) \cdot \boldsymbol{n}_{\boldsymbol{x}} \, dS(\boldsymbol{x}) = 0 \tag{B.5}$$

and

$$I_2 = \lim_{\epsilon \to 0} \int_{\partial B_\epsilon(\boldsymbol{y})} \big[\phi(\boldsymbol{y}) - \phi(\boldsymbol{x})\big]c(\boldsymbol{x})\nabla G(\boldsymbol{x}, \boldsymbol{y}) \cdot \boldsymbol{n}_{\boldsymbol{x}} \, dS(\boldsymbol{x}) = 0. \tag{B.6}$$

Notably, the governing Eq. (B.1) do not prohibit the existence of dipole or higher multipole singularities, but the regularity requirement $G(\cdot, \boldsymbol{y}) \in W^{1,1}(\Omega)$ forces $\nabla G(\cdot, \boldsymbol{y}) \in L^1(\Omega)$ and thus leaves only the monopole singularity [34,36], that is,

$$G(\boldsymbol{x}, \boldsymbol{y}) \sim \begin{cases} \ln\|\boldsymbol{x} - \boldsymbol{y}\|, & \text{for } d = 2, \\ \|\boldsymbol{x} - \boldsymbol{y}\|^{2-d}, & \text{for } d \geq 3. \end{cases} \quad \text{and} \quad \nabla G(\boldsymbol{x}, \boldsymbol{y}) \sim \begin{cases} \|\boldsymbol{x} - \boldsymbol{y}\|^{-1}, & \text{for } d = 2, \\ \|\boldsymbol{x} - \boldsymbol{y}\|^{1-d}, & \text{for } d \geq 3, \end{cases}$$

for $\boldsymbol{x}$ near the source point $\boldsymbol{y}$. Therefore, by the boundedness of $c(\boldsymbol{x})$ and $\nabla\phi(\boldsymbol{x})$, we have

$$\begin{aligned} |I_1| &= \lim_{\epsilon \to 0} \left| \int_{\partial B_\epsilon(\boldsymbol{y})} c(\boldsymbol{x})G(\boldsymbol{x}, \boldsymbol{y})\nabla\phi(\boldsymbol{x}) \cdot \boldsymbol{n}_{\boldsymbol{x}} \, dS(\boldsymbol{x}) \right| \leq \lim_{\epsilon \to 0} C \int_{\partial B_\epsilon(\boldsymbol{y})} |G(\boldsymbol{x}, \boldsymbol{y})| \, dS(\boldsymbol{x}) \\ &\leq \begin{cases} \lim\limits_{\epsilon \to 0} C|\partial B_\epsilon(\boldsymbol{y})| \ln\epsilon = \lim\limits_{\epsilon \to 0} C\epsilon \ln\epsilon = 0, & \text{for } d = 2, \\ \lim\limits_{\epsilon \to 0} C|\partial B_\epsilon(\boldsymbol{y})|\epsilon^{2-d} = \lim\limits_{\epsilon \to 0} C\epsilon^{d-1}\epsilon^{2-d} = 0, & \text{for } d \geq 3, \end{cases} \end{aligned}$$

where $C > 0$ is a generic constant. On the other hand, the continuity of the test function implies that there exists a constant $L > 0$ such that

$$|\phi(\boldsymbol{x}) - \phi(\boldsymbol{y})| \leq L\epsilon \quad \text{for } \boldsymbol{x} \in \partial B_\epsilon(\boldsymbol{y}).$$

Then, we arrive at

$$\begin{aligned} |I_2| &= \lim_{\epsilon\to 0}\left|\int_{\partial B_\epsilon} c(\boldsymbol{x})(\phi(\boldsymbol{x}) - \phi(\boldsymbol{y}))\nabla G(\boldsymbol{x},\boldsymbol{y})\cdot \boldsymbol{n}_{\boldsymbol{x}}\, ds\right| \leq \lim_{\epsilon\to 0} CL\epsilon \int_{\partial B_\epsilon(\boldsymbol{y})} |\nabla G(\boldsymbol{x},\boldsymbol{y})\cdot \boldsymbol{n}_{\boldsymbol{x}}|\, dS(\boldsymbol{x}) \\ &\leq \lim_{\epsilon\to 0} CL\epsilon |\partial B_\epsilon(\boldsymbol{y})|\epsilon^{1-d} = \lim_{\epsilon\to 0} CL\epsilon\epsilon^{d-1}\epsilon^{1-d} = 0 \end{aligned}$$

which completes the proof. □

References

[1] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, Nature 521 (7553) (2015) 436–444.
[2] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press, 2016.
[3] G.E. Karniadakis, I.G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, L. Yang, Physics-informed machine learning, Nat. Rev. Phys. 3 (6) (2021) 422–440.
[4] K. Azizzadenesheli, N. Kovachki, Z. Li, M. Liu-Schiaffini, J. Kossaifi, A. Anandkumar, Neural operators for accelerating scientific simulations and design, Nat. Rev. Phys. 6 (2024) 320-328.
[5] I.E. Lagaris, A. Likas, D.I. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, IEEE Trans. Neural Netw. 9 (5) (1998) 987–1000.
[6] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys. 378 (2019) 686–707.
[7] L.C. Evans, Partial Differential Equations, 19, American Mathematical Society, 2010.
[8] R.J. LeVeque, Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-state and Time-dependent Problems, SIAM, 2007.
[9] J. Han, A. Jentzen, W. E, Solving high-dimensional partial differential equations using deep learning, Proc. Natl. Acad. Sci. 115 (34) (2018) 8505–8510.
[10] D. Gilbarg, N.S. Trudinger, Elliptic Partial Differential Equations of Second Order, 224, Springer, 1977.
[11] B. Yu, E. Weinan, The deep Ritz method: a deep learning-based numerical algorithm for solving variational problems, Commun. Math. Stat. 6 (1) (2018) 1–12.
[12] J. Sirignano, K. Spiliopoulos, DGM: a deep learning algorithm for solving partial differential equations, J. Comput. Phys. 375 (2018) 1339–1364.
[13] L. Lu, P. Jin, G. Pang, Z. Zhang, G.E. Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, Nat. Mach. Intell. 3 (3) (2021) 218–229.
[14] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar, Fourier neural operator for parametric partial differential equations, (2020). arXiv preprint arXiv:2010.08895.
[15] N. Boullé, C.J. Earls, A. Townsend, Data-driven discovery of Green's functions with human-understandable deep learning, Sci. Rep. 12 (1) (2022) 4824.
[16] C.R. Gin, D.E. Shea, S.L. Brunton, J.N. Kutz, DeepGreen: deep learning of Green's functions for nonlinear boundary value problems, Sci. Rep. 11 (1) (2021) 21614.
[17] H. Melchers, J. Prins, M. Abdelmalik, Neural Green's operators for parametric partial differential equations, (2024). arXiv preprint arXiv:2406.01857.
[18] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, A. Anandkumar, Neural operator: learning maps between function spaces with applications to PDEs, J. Mach. Learn. Res. 24 (89) (2023) 1–97.
[19] K. Shukla, J.D. Toscano, Z. Wang, Z. Zou, G.E. Karniadakis, A comprehensive and FAIR comparison between MLP and KAN representations for differential equations and operator networks, Comput. Methods Appl. Mech. Eng. 431 (2024) 117290.
[20] A. Kopaničáková, G.E. Karniadakis, DeepONet based preconditioning strategies for solving parametric linear systems of equations, SIAM J. Sci. Comput. 47 (1) (2025) C151–C181.
[21] E. Zhang, A. Kahana, A. Kopaničáková, E. Turkel, R. Ranade, J. Pathak, G.E. Karniadakis, Blending neural operators and relaxation methods in PDE numerical solvers, Nat. Mach. Intell. 6 (2024) 1303–1313.
[22] C. Cui, K. Jiang, Y. Liu, S. Shu, A hybrid iterative neural solver based on spectral analysis for parametric PDEs, J. Comput. Phys. 538 (2025) 114165.
[23] J. Hu, P. Jin, A hybrid iterative method based on MIONet for PDEs: theory and numerical examples, Math. Comput. 95 (2025) 1327–1359.
[24] Y. Lin, Y.J. Lee, J. Jia, Green multigrid network, (2024). arXiv preprint arXiv:2407.03593.
[25] G. Lin, P. Hu, F. Chen, X. Chen, J. Chen, J. Wang, Z. Shi, BInet: learning to solve partial differential equations with boundary integral networks, CSIAM Trans. Appl. Math. 4 (2) (2023) 275–305.
[26] Y. Teng, X. Zhang, Z. Wang, L. Ju, Learning Green's functions of linear reaction-diffusion equations with application to fast numerical solver, in: Mathematical and Scientific Machine Learning, PMLR, 2022, pp. 1–16.
[27] J. Ji, L. Ju, X. Zhang, Deep surrogate model for learning Green's function associated with linear reaction-diffusion operator, Int. J. Numer. Anal. Model. 21 (5) (2024) 697–715.
[28] T. Hu, B. Jin, Z. Zhou, Solving elliptic problems with singular sources using singularity splitting deep Ritz method, SIAM J. Sci. Comput. 45 (4) (2023) A2043–A2074.
[29] Z. Li, H. Zheng, N. Kovachki, D. Jin, H. Chen, B. Liu, K. Azizzadenesheli, A. Anandkumar, Physics-informed neural operator for learning partial differential equations, ACM/JMS J. Data Sci. 1 (3) (2024) 1–27.
[30] W. Hao, R.P. Li, Y. Xi, T. Xu, Y. Yang, Multiscale neural networks for approximating Green's functions, (2024). arXiv preprint arXiv:2410.18439.
[31] F. Chen, J. Huang, C. Wang, H. Yang, Friedrichs learning: weak solutions of partial differential equations via deep learning, SIAM J. Sci. Comput. 45 (3) (2023) A1271–A1299.
[32] N. Boullé, Y. Nakatsukasa, A. Townsend, Rational neural networks, Adv. Neural Inf. Process. Syst. 33 (2020) 14243–14253.
[33] K. Wimalawarne, T. Suzuki, S. Langer, Learning Green's function efficiently using low-rank approximations, (2023). arXiv preprint arXiv:2308.00350.
[34] M. Grüter, K.-O. Widman, The Green function for uniformly elliptic equations, Manuscr. Math. 37 (3) (1982) 303–342.
[35] S. Kim, G. Sakellaris, Green's function for second order elliptic equations with singular lower order coefficients, Commun. Partial Differ. Equations 44 (3) (2019) 228–270.
[36] G. Dolzmann, S. Müller, Estimates for Green's matrices of elliptic systems by L^p theory, Manuscr. Math. 88 (1995) 261–273.
[37] D. Mitrea, I. Mitrea, On the regularity of Green functions in Lipschitz domains, Commun. Partial Differ. Equations 36 (2) (2010) 304–327.
[38] Y. Nakatsukasa, O. Sète, L.N. Trefethen, The AAA algorithm for rational approximation, SIAM J. Sci. Comput. 40 (3) (2018) A1494–A1522.
[39] I. Stakgold, M.J. Holst, Green's Functions and Boundary Value Problems, John Wiley & Sons, 2011.
[40] S.C. Brenner, L.R. Scott, Math. Theory Finite Element Methods, 3, Springer, 2008.
[41] L. Zhang, T. Luo, Y. Zhang, Z.-Q.J. Xu, Z. Ma, et al., MOD-Net: a machine learning approach via model-operator-data network for solving PDEs, Commun. Comput. Phys. 32 (2) (2022) 299–335.
[42] R.S. Varga, Iterative analysis, New Jersey 322 (1962) 24.
[43] G.H. Golub, C.F. Van Loan, Matrix Computations, JHU Press, 2013.
[44] L.N. Trefethen, D. Bau, Numerical Linear Algebra, 181, SIAM, 2022.
[45] Y. Saad, Iterative Methods for Sparse Linear Systems, SIAM, 2003.

[46] U. Trottenberg, C.W. Oosterlee, A. Schuller, Multigrid Methods, Academic Press, 2001.
[47] A. Toselli, O. Widlund, Domain Decomposition Methods-Algorithms and Theory, 34, Springer Science & Business Media, 2004.
[48] Z.-Q.J. Xu, L. Zhang, W. Cai, On understanding and overcoming spectral biases of deep neural network learning methods for solving PDEs, (2025). arXiv preprint arXiv:2501.09987.
[49] Z. Li, D. Xiao, Z. Lai, W. Wang, Neural preconditioning operator for efficient PDE Solves, (2025). arXiv preprint arXiv:2502.01337.
[50] A. Klawonn, M. Lanser, J. Weber, Learning adaptive coarse basis functions of FETI-DP, J. Comput. Phys. 496 (2024) 112587.
[51] W. Xinyi, C. Jingrun, D. Rui, A label-free hybrid iterative numerical transferable solver for partial differential equations, CSIAM Trans. Appl. Math. 5 (2024) 712–734.
[52] A.G. Baydin, B.A. Pearlmutter, A.A. Radul, J.M. Siskind, Automatic differentiation in machine learning: a survey, J. Mach. Learn. Res. 18 (153) (2018) 1–43.
[53] Y.-H. Tseng, T.-S. Lin, W.-F. Hu, M.-C. Lai, A CUSP-capturing PINN for elliptic interface problems, J. Comput. Phys. 491 (2023) 112359.
[54] W.-F. Hu, T.-S. Lin, M.-C. Lai, A discontinuity-capturing neural network with categorical embedding and its application to anisotropic elliptic interface problems, J. Comput. Phys. 530 (2025) 113905.
[55] K. Hornik, M. Stinchcombe, H. White, Multilayer feedforward networks are universal approximators, Neural Netw. 2 (5) (1989) 359–366.
[56] I.E. Lagaris, A.C. Likas, D.G. Papageorgiou, Neural-network methods for boundary value problems with irregular boundaries, IEEE Trans. Neural Netw. 11 (5) (2000) 1041–1049.
[57] N. Rahaman, A. Baratin, D. Arpit, F. Draxler, M. Lin, F. Hamprecht, Y. Bengio, A. Courville, On the spectral bias of neural networks, in: International Conference on Machine Learning, PMLR, 2019, pp. 5301–5310.
[58] Z.-Q.J. Xu, Y. Zhang, T. Luo, Overview frequency principle/spectral bias in deep learning, Commun. Appl. Math. Comput. 7 (2024) 827–864.
[59] T. Hsing, R. Eubank, Theoretical Foundations of Functional Data Analysis, with an Introduction to Linear Operators, 997, John Wiley & Sons, 2015.
[60] P. Frauenfelder, C. Schwab, R.A. Todor, Finite elements for elliptic problems with stochastic coefficients, Comput. Methods Appl. Mech. Eng. 194 (2-5) (2005) 205–228.
[61] I. Steinwart, C. Scovel, Mercer's theorem on general domains: on the interaction between measures, kernels, and RKHSs, Constructive Approximation 35 (2012) 363–417.
[62] G.B. Folland, Real Analysis: Modern Techniques and Their Applications, John Wiley & Sons, 1999.
[63] F. Hartmann, Green's Functions and Finite Elements, Springer Science & Business Media, 2012.
[64] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, (2014). arXiv preprint arXiv:1412.6980.
[65] Y. Saad, M.H. Schultz, GMRES: a generalized minimal residual algorithm for solving nonsymmetric linear systems, SIAM J. Sci. Stat. Comput. 7 (3) (1986) 856–869.
[66] T. Myint-U, L. Debnath, Linear Partial Differential Equations for Scientists and Engineers, Springer, 2007.
[67] A. Henrot, Extremum Problems for Eigenvalues of Elliptic Operators, Springer, 2006.